Autonomous Robotic Design for Strategic Adversarial Games

A Major Qualifying Project By:

Luca Dang, Computer Science & Mathematical Science

Elene Kajaia, Computer Science & Robotics Engineering

Vishesh Konduru, Robotics Engineering

Graham Mack, Robotics Engineering

Leo Riesenbach, Computer Science & Electrical and Computer Engineering

Mason Sakai, Robotics Engineering

Dylan Schmit, Computer Science & Robotics Engineering

Submitted to

Griffin Tabor, Robotics Engineering

Bradley Miller, Robotics Engineering

Xinming Huang, Electrical and Computer Engineering

This report represents the work of WPI undergraduate students submitted to the faculty as evidence of completing a degree requirement. WPI routinely publishes these reports on their website without editorial or peer review

Abstract

Worldwide, over 10,000 high school robotics teams compete in a fast-paced, annual competition using teleoperated robots. Our goal is to demonstrate that a fully autonomous robot can compete at this same level. Autonomous performance in dynamic, adversarial environments remains a major challenge in robotics, requiring systems that can continuously interpret the environment and respond in real time. This project aims to develop a fully autonomous system that integrates multi-sensor perception, real-time state estimation, and a strategic decision-making model to determine how to respond to an opponent. By translating advanced robotics into a familiar and engaging context, we aim to inspire the next generation of engineers.

Contributions

Section	Primary Author(s)
Introduction	Leo Riesenbach
Literature Review	
VEX Robotics Competition	Leo Riesenbach
VEX AI Robotics Competition	Dylan Schmit
Previous WPI Project: ASGARD	Vishesh Konduru
Robot Operating System 2	Leo Riesenbach, Kylie Herbstzuber
Model Predictive Path Integral Controller	Dylan Schmit
Adaptive Monte Carlo Localization	Luca Dang, Graham Mack
Computer Vision	Luca Dang
Reinforcement Learning for Adversarial Games	Elene Kajaia
Methodology and Results	
Software System Design	Mason Sakai, Leo Riesenbach
Simulation	Kylie Herbstzuber
Robot Design	Vishesh Konduru
Robot Electronics	Leo Riesenbach
Computer Vision - Image Processing	Vishesh Konduru
Computer Vision - YOLO	Dylan Schmit
Computer Vision - Unified Game State Vision Model	Luca Dang
Computer Vision - YOLO Block Position Mapping	Vishesh Konduru
Strategy AI	Elene Kajaia
Future Work	
Robot Electronics	Leo Riesenbach
Computer Vision	Luca Dang
Strategy AI	Elene Kajaia

Table of Contents

Abstract **2**
Contributions **3**
Table of Contents **4**
Introduction **7**
Literature Review **8**
VEX Robotics Competition 8
VEX AI Robotics Competition 8
Previous WPI Project: ASGARD 10
Robot Operating System 2 11
Model Predictive Path Integral Controller 12
Adaptive Monte Carlo Localization 13
Visual Object Detection 14
Reinforcement Learning for Adversarial Games 17
Methodology and Results **19**
Software System Design 19
Motion Planning and Localization 21
Action Server 21
Simulation 24
Optimizing for Performance 24
Rolling Friction Bug 26
World Interaction 26
Robot Design 28
Drivetrain 29
Intake and Scoring 30
Block Storage 31
Robot Electronics 32
VEX V5 Brain/Motors 33
Communications Board 34
Power Board 36
Cameras 39
I2C Drivers 41
Computer Vision - Image Processing 42
Illumination Correction 43
Computer Vision - YOLO 45
YOLO Block Detection Model 45
Roboflow Dataset 46

YOLO ROS 2 Node 47
Computer Vision - Unified Game State Vision Model 48
Dataset 48
Object Detection 51
Discrete Occupancy Prediction 51
Dataset Preprocessing 51
Neural Network Overview 52
Feature Extractor 53
Camera Fusion 55
Transformer Encoder 58
Evaluation 63
3D Coordinate Prediction 64
Iterative Refinement Detection Head 65
Hungarian Loss 66
Evaluation 67
Computer Vision - YOLO Block Position Mapping 69
Ground Plane Projection 70
Learned Residual Neural Network Correction 71
Strategy AI 74
Iteration 1 - Discrete Simulation and Synchronous Training 75
Simulation 75
Training 77
Iteration 2 - Improved Discrete Simulation and Synchronous Training 80
Simulation 80
Training 81
ResNet architecture 81
General Advantage Estimation (GAE) 81
League Players 82
Multithreaded Rollout Generation 82
Evaluation 83
Iteration 3 - Continuous Simulation and Asynchronous Training 83
Simulation 84
Field Representation and Model 85
Training 87
Sample Reuse 88
Staleness 90
Results 91
Future Work **92**

Robot Electronics 92
Computer Vision - Unified Game State Vision Model 93
Strategy AI 94
References **95**

Introduction

Autonomous decision-making in adversarial environments remains a challenge in robotics. We aim to tackle this challenge by building an autonomous robot to compete in the 2025-2026 VEX Robotics game, "Push Back." This game is typically played by human tele-operators driving robots, with only limited, interaction-free, autonomous periods. Our goal is to design and build a robotic system that can autonomously play against a human player with full dynamic interaction, using only onboard sensing and computation. Building a dynamically autonomous robot that fits roughly within the VEX Robotics Competition ruleset is a challenge, due primarily to the difficulty of determining the game state from within the robot, the strategic complexity of the game and the challenge of integrating a complicated software and embedded hardware system.

Our design to address these challenges combines perception, strategy, pathing and hardware control. We tackled perception using a transformer-based vision model to determine the overall field state from only the onboard camera views. The robot's strategic planning uses a reinforcement learning model to interpret the current game state into an optimal next action, which is broken down by the pathing and action components of the system based around Model Predictive Path Integral controller and Adaptive Monte Carlo Localization. Finally, the robot hardware executes these actions through a network of sensors and actuators via custom-designed circuit boards.

The work for this project focuses on solving each of the key challenges in a modular way, such that they can be tested individually and integrated into a full system. We aim to demonstrate that it is possible for autonomous robots to effectively and dynamically compete in strategic adversarial environments against human-operated opponents.

Literature Review

VEX Robotics Competition

The VEX Robotics Competition provides a platform for student teams to design, build, and program robots for structured, competitive matches. Games are played on a 12’ × 12’ field, with two opposing alliances of two teams competing across the same set of scoring objects. The 2025–2026 game which we are designing for is called Push Back, and features 88 blocks of two different colors. Teams must build robots to place the blocks corresponding to their color into the horizontal-tube-shaped goals. Goals vary in height and length, so robots must be able to eject blocks at different heights (VEX Robotics Inc., 2025). The dense placement of objects, presence of opponent robots, and limited match time creates a highly dynamic environment, which presents significant challenges for navigation, path planning, and strategic decision-making.

The typical ruleset for building and designing a robot includes that robots must be powered and controlled using the VEX V5 electronics system, and built out of the structural and mechanical elements of the VEX V5 system. Robots must fit into a size limit of 18x18x18" at the beginning of the match, although limited size expansion after the match begins is permitted. They are also allowed only a limited number of motors, and the only motors allowed for use are the VEX V5 Smart Motors, in either the 11W or 5.5W variety (VEX Robotics Inc., 2025).

VEX AI Robotics Competition

The VEX AI Robotics Competition is a variant of the VEX Robotics Competition. It was initially launched for the 2020-2021 season and temporarily discontinued after the 2021-2022 season. It was relaunched with a more open ruleset for the 2023-2024 season and beyond. For the 2025-2026 VEX AI Robotics Competition, the game is largely the same, with some

modifications: Instead of two alliances of two teams each, matches are played by two teams. Each team must build one robot that fits with a 15" sizing cube, and one that fits in a 24" sizing cube. Restrictions around custom manufactured mechanical parts and electronics beyond the VEX V5 system are also significantly loosened. Instead of a 15 second autonomous period and a 1:45 driver controlled period, the entire match is autonomous with a 15 second isolation period where teams must stay on their side of the field, and a 1:45 interaction period, where they can interact with opponent robots. In addition to being fully autonomous, the initial field setup and scoring rules are altered to be more challenging and conducive to fully autonomous play (VEX Robotics Inc., 2025).

While there is not much published work from the post-2023 version of the competition, team BLRSAI published their approach for the 2021-2022 game Tipping Point. Their system consisted of 2 main AI components: a reinforcement learning model trained in simulation to control robot movement, an object detection to track game pieces, and a communication system between the VEX V5 brain and an onboard Jetson Nano. Their reinforcement learning model was trained using a Unity simulation. Their reinforcement learning model takes the positions of all robots on the field, the 10 closest known game objects, and the time in since the start of the game in seconds and outputs a linear and angular velocity to control the robot directly. They found that using reinforcement learning to control the robot's movement directly did not lead to defensive strategies emerging as they had anticipated, and suggested that changing the reward function, simulation, or training for longer could cause these behaviors to emerge. The robot was able to collect game objects well both in simulation and real life, but the authors recommend teams use a more traditional approach to path planning/control for a future competitive robot. For object detection they used a YOLO model, YOLOv5s, combined with GRIP pipeline

employing HSV filtering to omit background noise. They used an Intel Realsense depth camera to estimate the pose of detected objects relative to the robot. They found their system to be effective, but recommended that future teams get more data and experiment with different computer vision model architectures better suited for detecting small objects (Zietek, et al., 2022).

Previous WPI Project: ASGARD

The ASGARD (Adversarial Strategic Games and Robotic Design) project was a previous Major Qualifying Project at Worcester Polytechnic Institute aimed at developing software for a fully autonomous robotic competitor for the 2017-2018 VEX U game, "In the Zone" (Tabor, 2018). The project focused on four main capabilities: real-time motion planning, long-term localization, object detection, and adversarial strategic planning.

For navigation and localization, the team utilized the Robot Operating System (ROS). They fused wheel encoder odometry with 2D LIDAR scan data using Google Cartographer, successfully preventing the buildup of positional error over extended operating sessions. Their navigation system relied on the ROS teb_local_planner combined with a standard global planner to navigate the cluttered field.

For object detection, the ASGARD team employed a customized TensorFlow object detection package to track game pieces. While they achieved a mean average precision (mAP) of 0.9 during training, they encountered real-time performance issues when deploying the model on the physical robot.

To handle high-level decision-making, the team built a discrete state simulator of the VEX game and implemented a modified reinforcement learning approach called

"Confidence-interval Q-learning". By selecting actions based on the lower bound of a Q-value's confidence interval rather than a standard greedy approach, their simulated agent converged to a nearly 80% win rate against a naive reflex agent. Ultimately, while the team successfully developed these individual software components, time constraints prevented them from fully integrating and testing the complete system during an actual physical match, leaving full hardware integration as a recommendation for future teams.

Robot Operating System 2

Robot Operating System 2 (ROS 2) is a popular open source software framework for robotics (Macenski et al., 2022). It models software architectures as a graph of nodes, which communicate through a middleware to send messages and actions through a publish/subscribe topic model (Macenski et al., 2023). Many of the most critical robotics functions, such as common algorithms for controls, path planning, and localization are available as standardized libraries of nodes, reducing implementation effort. The decentralized architecture also aids in making systems highly modular and their components interchangeable. This is a major benefit of the ROS 2 design, as it enables maximal code reuse with minimal changes between simulated environments and real world, as well as in different hardware configurations (Macenski et al., 2023).

ROS 2 is also integrated with a simulation environment called Gazebo, which provides physics simulations, sensor emulation, and joint control plugins. This tool allows testing most of a robot system's software stack in the simulated environment without requiring the fully integrated hardware system, which greatly accelerates and simplifies development efforts. Since

ROS 2's decentralized architecture allows modular replacement of nodes, swapping out "real" sensor nodes for ones emulated using Gazebo is an effective testing strategy.

Model Predictive Path Integral Controller

ROS 2's primary navigation project, Nav2, contains many control algorithms for both differential drive and omnidirectional robots. One of the most effective included algorithms is the Model Predictive Path Integral Controller (MPPI). MPPI falls under the umbrella term of Model Predictive Control (MPC), where instead of purely reacting to changes in a control variable like PID control, the controller incorporates a model of the system.

MPPI is based on stochastic trajectory optimization. Many candidate trajectories are sampled based on an initial trajectory by adding Gaussian perturbations to the control inputs and using the model to update the trajectory. The total cost for each sampled trajectory is then calculated. Each control input of the original trajectory is updated using an exponentially weighted average of all sampled trajectories, with lower cost trajectories weighted higher. The process of generating and scoring many trajectories can be parallelized and processing can be sped up using a GPU, although CPU-only performance is also often sufficient for practical use. MPPI exhibits many behaviors that make it derisable in dynamic environments, such as the ability to predictively "back out" when faced with obstacles, making it less likely to get stuck. It is also generally considered the best choice of Nav2 controller for omni-directional robots (Macenski, et al., 2023).

Adaptive Monte Carlo Localization

Adaptive Monte Carlo Localization (AMCL) is a probabilistic particle-filter-based algorithm for mobile robot localisation in known 2D environments. It represents the robot's belief about its pose as a set of weighted particles and recursively updates this belief using odometry (motion model) and sensor observations (measurement model, typically laser range finders). Unlike deterministic filters such as the Extended Kalman Filter (EKF), AMCL can handle multi-modal distributions, global localisation from scratch, and the "kidnapped robot" problem. The "adaptive" aspect introduced via Kullback-Laibler Divergence (KLD) sampling dynamically adjusts the number of particles to maintain a statistical bound on approximation error, which can improve computational efficiency while maintaining accuracy.

AMCL stemmed from Monte Carlo Localisation (MCL), a particle filter algorithm used for robot localization. "Monte Carlo Localization for Mobile Robots" ICRA (Dellaert, 1999) first introduced MCL. Their approach represented the posterior pose distribution with a fixed number of weighted particles which are updated via importance sampling and resampling. This paper showed that particle filters could robustly track a robot's pose even in highly symmetric or ambiguous indoor spaces where Kalman-filter-based methods failed due to multi-modality.

"Robust Monte Carlo Localization for Mobile Robots" (Thrun, 2001) refined MCL, introducing methods to handle global localisation and the kidnapped-robot problem by adding random "recovery" particles when pose confidence dropped. This showed significant robustness in real-world settings. These early MCL variants relied on fixed and often large particle counts, leading to unnecessary computation overhead even when the robot is well localised. .

"KLD-Sampling: Adaptive Particle Filters" (Fox, 2001) introduced AMCL. Instead of using a fixed sampled size, KLD-sampling computes the minimum number of particles M

Required at each step such that, with probability $1-\sigma$, the KL divergence (or distance) between the true posterior and sample-based approximation is bounded by a small ϵ:

$$M \approx \frac{k-1}{2\epsilon}(1-\frac{2}{9(k-1)}+\sqrt{\frac{2}{9(k-1)}}z_{1-\sigma})^3$$

Where k is the number of occupied bins in a discrete approximation of the posterior distribution. Fundamentally, this statistical guarantee allows the filter to use a tiny amount of articles (often <100) when the pose is certain and many more only if uncertainty is high.

Visual Object Detection

Object detection is a fundamental problem in computer vision. It aims to locate objects of interest within the image and predict their categories and/or corresponding bounding boxes. The past decade has seen rapid progress of 2D object detection. Various state-of-the-art approaches have been introduced over the years: YOLO (Redmon, 2015) and its many iterations, Faster R-CNN (Ren, 2015), RetinaNet (Lin, 2017), and FCOS (Tian, 2019). However, 2D information is not enough for an intelligent agent to perceive the 3D world. In order to react in an adversarial setting, the agent must have accurate 3D information of objects around it to make intelligent decisions (Wang, 2021). 3D object detection research and model development is generally split into 2 main branches: LiDAR-based and monocular (camera)-based.

Traditional 2D object detection relied on hand-crafted features, such as the Viola-Jones detector (Viola, 2001) for face detection using Haar-like features and sliding windows, followed by Histogram of Oriented Gradient (Dalal, 2005), and Deformable Part Models (Felzenszwalb, 2008). These methods were computationally intensive and lacked robustness to scale, occlusion and different lighting (Zhao, 2019). The deep learning methods quickly took over, starting with

AlexNet (Krizhevsky, 2012), enabling end-to-end feature learning at scale. This led to the R-CNN family, which combined regional proposals with CNN features. Fast R-CNN (Girshick, 2015) introduced ROI pooling for efficiency, and Faster R-CNN added Region Proposal Network for near-real-time performance. One-stage detectors emerged for speed, such as YOLO formulating detection as a regression problem (Redmon, 2015). SSD (Liu, 2015) used multi-scale feature maps. RetinaNet (Lin, 2017) addresses class imbalance with focal loss, and FCOS (Tian, 2019) introduced anchor-free detection. Recent Transformer-based models like DETR (Carion, 2020) eliminated anchors and Non-Maximum Suppression (Neubeck, 2006) entirely, modelling detection as a set-prediction task with Hungarian loss (Carion, 2020). These advancements dramatically improved accuracy on benchmarks like MS COCO (Lin, 2014) and VOC (Everingham, 2010), with modern YOLO versions v8-v10 (Jocher, 2023; Wang 2024; Wang 2024) balancing speed and precision for real-world robotics applications.

While 2D detection suffices for image understanding, real-world agents (robots, autonomous vehicles) require 3D localisation, depth, orientation, and velocity for planning and control. Early 3D methods extended 2D detectors by regression over additional attributes (e.g., depth, 3D box dimensions) but suffered from depth ambiguity in perspective images, which computer vision researchers split into sensor-driven paradigms to overcome: LiDAR-based & camera-based.

The alternative approach to object detection with cameras is using LiDARs. LiDAR provides direct 3D point clouds, allowing high-precision geometric reasoning independent of lighting conditions. Early approaches used hand-crafted features or voxelisation. VoxelNet (Zhou, 2017) introduced end-to-end learning with neural networks by voxelising point clouds and applying 3D convolutions. PointPillars (Lang, 2019) projects points into vertical pillars for

faster 2D-like processing. Later works like PV-RCNN (Shi, 2021) fused point-based and voxel-based features, while CenterPoint (Yin, 2021) adopted anchor-free center-based detection for faster inference. Recent state-of-the-arts include lightweight backbones like Pillar R-CNN (Shi, 2023), dense-layer adaptations for edge devices and self-supervised methods like LISO (Baur, 2024) that train on unlabeled sequence via trajectory regularisation. On KITTI (Geiger, 2012) and nuScenes (Caesar, 2020) benchmarks, LiDAR methods achieve extremely high average precision.

Monocular approaches use only RGB cameras, making them cost-effective for deployment. Early methods like Mono3D (Kumar, 2025), Deep3DBox (Mousavian, 2017), FCOS3D regressed 3D attributes directly from 2D features but struggled with depth estimation. The field shifted to Bird's-Eye-View (BEV) representation: transforming perspective image features into a top-down orthographic grid that aligns naturally with planning and control.

In a multi-camera setting such as autonomous driving, many early BEV methods processed single-camera predictions first, then fused them across cameras using probabilistic formulations. For example, Lift-Splat-Shoot (Philion, 2020) predicts a probabilistic depth distribution for each pixel in a single image, "lifts" the 2D image features into a 3D frustum of weighted points, and "splats" them into a shared BEV grid. This probabilistic lifting allows soft fusion across overlapping camera views in the BEV space, but is unstable for diverse scenes with occlusions.

Modern Transformer-based approaches such as BEVFormer (Li, 2022) and DETR3D (Wang 2021) instead used internal learned fusion directly within the neural network. BEVFormer uses grid-shaped learnable positional BEV queries that interact with multi-camera image features through spatial cross-attention, specifically deformable for efficiency. Each BEV query

adaptively samples and aggregates relevant regions from all available camera views in a single unified process. Temporal modeling is also added via temporal attention which fuses historical BEV features. BEV-based perception powers real-world autonomous systems. Tesla's vision-only Full Self-Driving stack famously relies on multi-camera inputs processed through BEV and Transformer architectures, rejecting LiDAR.

Reinforcement Learning for Adversarial Games

Before the advent of modern artificial intelligence and physical AI, the majority of robotics policies depended on hand-tuned logic, usually employing simple decision trees and conditional statements to model autonomous navigation. Furthermore, Minimax algorithms work by simulating a tree of future possibilities to minimize the potential loss in a "worst-case" scenario. However, in games where the number of possible outcomes is very high, it is difficult and in most cases unusual to use them due to the computational complexity (Campbell, 1983).

Recently, the field has shifted to using Neural Networks with imitation learning. Such an approach allows an agent to determine actions based on gathered data to learn optimal behaviors (Ross, 2010). However, this approach is often unfeasible to implement without extensive data from high-level players. Therefore, the field has moved toward Reinforcement Learning architectures, where the agent learns independently through rewards gained by playing the game.

Reinforcement learning algorithms are generally divided into two categories: value-based networks, and policy-based networks. In a value-based network, the model outputs a value for each potential action, where a higher value indicates a better move to take. Conversely, a policy network determines which action to take based on a probability distribution. Value networks, such as Q-learning, often fail in environments with an infinite number of possible actions, such

as a continuous action space (Watkins, 1992). Because of this limitation, policy network that utilizes an actor and a critic are more widely used for adversarial games.

Policy gradient methods originated from the REINFORCE algorithm, which is considered the vanilla policy gradient method. In this approach, the agent plays a game and collects raw rewards at each step, with the model being trained at the end using Monte Carlo methods (Williams, 1992). Since then, various improvements have been introduced. One significant development was adding a baseline to REINFORCE, which helps the algorithm stabilize the direction of the gradient for the neural network parameters. The field eventually moved toward Actor-Critic methods. Instead of waiting for a game to finish, the model uses an actor to provide a distribution of actions and a critic to estimate the cumulative future rewards from the following timesteps. Subsequent variations like Trust Region Policy Optimization (TRPO) were designed to ensure that the new policy does not deviate too far from the previous one, preventing drastically changing gradients that could destabilize training (Schulman, 2017). The most well known and widely used evolution of this is Proximal Policy Optimization (PPO) (Schulman, 2017). This method introduced a clipping function to ensure that policy updates stay within a certain threshold, making the training process much more robust and efficient.

A significant surge in interest for using reinforcement learning in adversarial games began in 2016 when Google DeepMind defeated the world champion in Go using AlphaGo (Silver, 2016). AlphaGo utilized policy and value networks alongside Monte Carlo Tree Search to look ahead at future states of the game. In reinforcement learning algorithms for adversarial games, it is crucial to consider the opponent the agent is training against. AlphaGo addressed this by simulating the opponent using the same agent, a technique known as self-play. Unlike AlphaGo, AlphaStar used a "league" of players, consisting of both current and previous versions

of the model, to simulate games and train the agent (Mathieu, 2023). Later, OpenAI also applied reinforcement learning to gaming with OpenAI Five, which was designed to play DOTA 2. OpenAI Five employed Long Short Term Memory (LSTM) neural networks combined with PPO and the Generalized Advantage Estimation (GAE) function. OpenAI Five achieved a 99.4% win rate over 7,000 games (OpenAI, 2023).

Methodology and Results

Software System Design

The robot software is a complex system, involving several different hardware drivers, a mix of classical robotics techniques for some of the functionality, and custom AI models for others. All of the subsystems are internally represented by ROS 2 nodes and integrated across the ROS 2 middleware through topics and actions. We started by modelling out the high-level subsystems and how they would interact to make up the full robot. This high-level subsystem architecture design is visible in Figure 1.

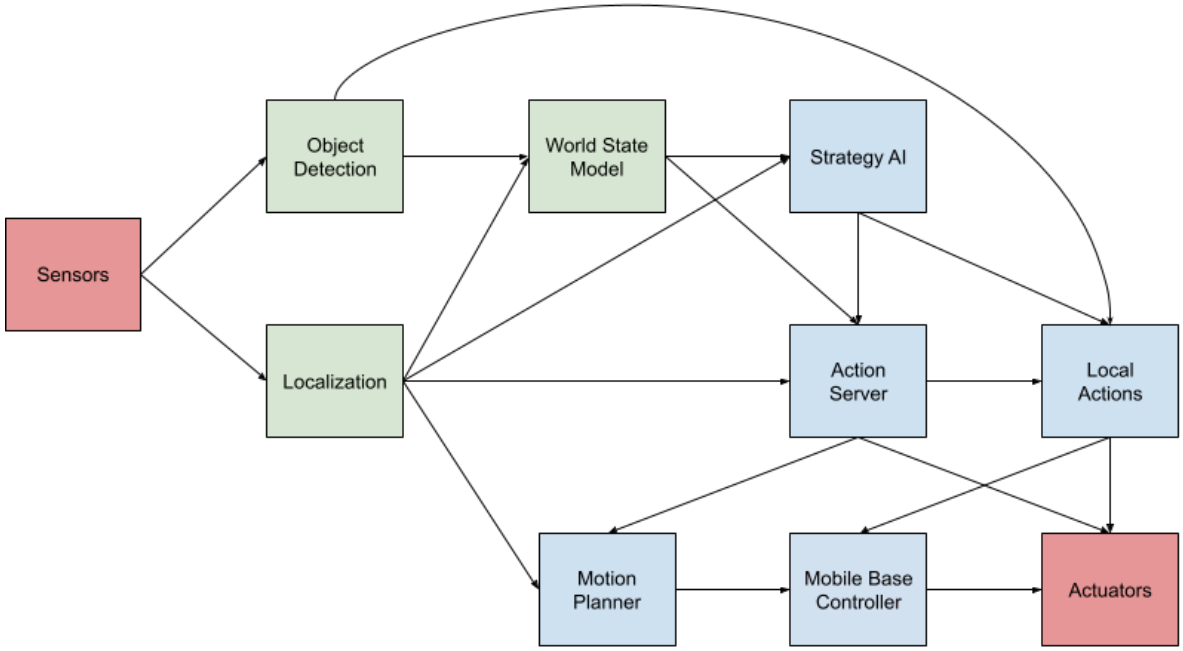

Figure 1: Software Subsystem Architecture Design

The system architecture design can be broken up into hardware drivers (highlighted in red), sensing (highlighted in green), and execution (highlighted in blue). The hardware driver components consist of inputs from the various sensors, and outputs to control the robot's mobile base and its object manipulation actuators. The sensing nodes use the sensor values to determine the robots position on the field through localization techniques, as well as the dynamic position of all of the game elements and the opponent robot, then fuses all this information into a single, coherent, constantly updating world state. The execution nodes take this world state generated by the sensing nodes and use it to generate high-level actions and low-level commands for the actuators to perform for the robot to play the game.

Motion Planning and Localization

The motion planning makes use of the popular ROS framework, Navigation 2 (Nav2), using its Model Predictive Path Integral (MPPI) controller and Adaptive Monte-Carlo Localizer (AMCL) packages. These handle the navigation and control of the robot, with the exception of local actions. AMCL utilizes LiDAR, wheel odometry, and an IMU for localization.

Action Server

The Action Server is responsible for taking abstract high-level strategic tasks and using Nav2 to accomplish those tasks. The core Action Server is a single node that manages all Nav2-related actions and handles execution mutual-exclusivity to prevent conflicting commands. The actions are implemented through a class that the Action Server manages, and each action registers with ROS 2 as they would normally through the Server's node. When an action is started, it locks a mutex that ensures exclusivity and signals an action is actively running. If another action is called, the current action gets cancelled before the next begins, and any active action can be cancelled through a service. There are two ways actions can be implemented in the action server: using multiple Nav2 API calls, and behavior trees.

Starting with API calls, both methods use Nav2's Basic Commander API, which can, among other things, execute go-to-point commands and monitor their progress. Simple actions such as scoring, parking, using the loaders, and driving to poses each consist of 2-3 API calls.

Complex actions like the "pick up blocks in an area" action work through a simple control loop. For this action, Nav2 is commanded to drive to the center of the search area, and waits for the robot to enter the area. If the robot spots a block in the area, the current navigation is canceled and Nav2 is commanded to drive to the block to pick it up. This is repeated until the

specified limit is reached or no blocks are spotted in the area, at which point the action is complete. Critically, this method is entirely implemented in Python, though complex tasks done this way can also utilize custom behavior trees.

Behavior trees use the BehaviorTree.CPP library, and are XML-defined hierarchical state machines which Nav2 utilizes for executing movement tasks. The default trees Nav2 provides handle path planning, path following, and recovery from errors or getting stuck. The default trees also define the rates at which replanning happens and how many times a recovery can occur before failure. The Basic Commander API allows custom behavior trees to be used with any relevant action, so actions can be defined in code with a single go-to-point call and status checks. Actions made with behavior trees have full control over the navigator, including access to variables like the path and goal and the ability to change recovery or add extra behaviors. Additionally, since behavior trees can be specified for any API call, other actions can run a behavior tree action with no additional code. The "pickup while driving" action was made using a behavior tree (shown in Figure 2), which required the creation of a Nav2 plugin for custom BT nodes.

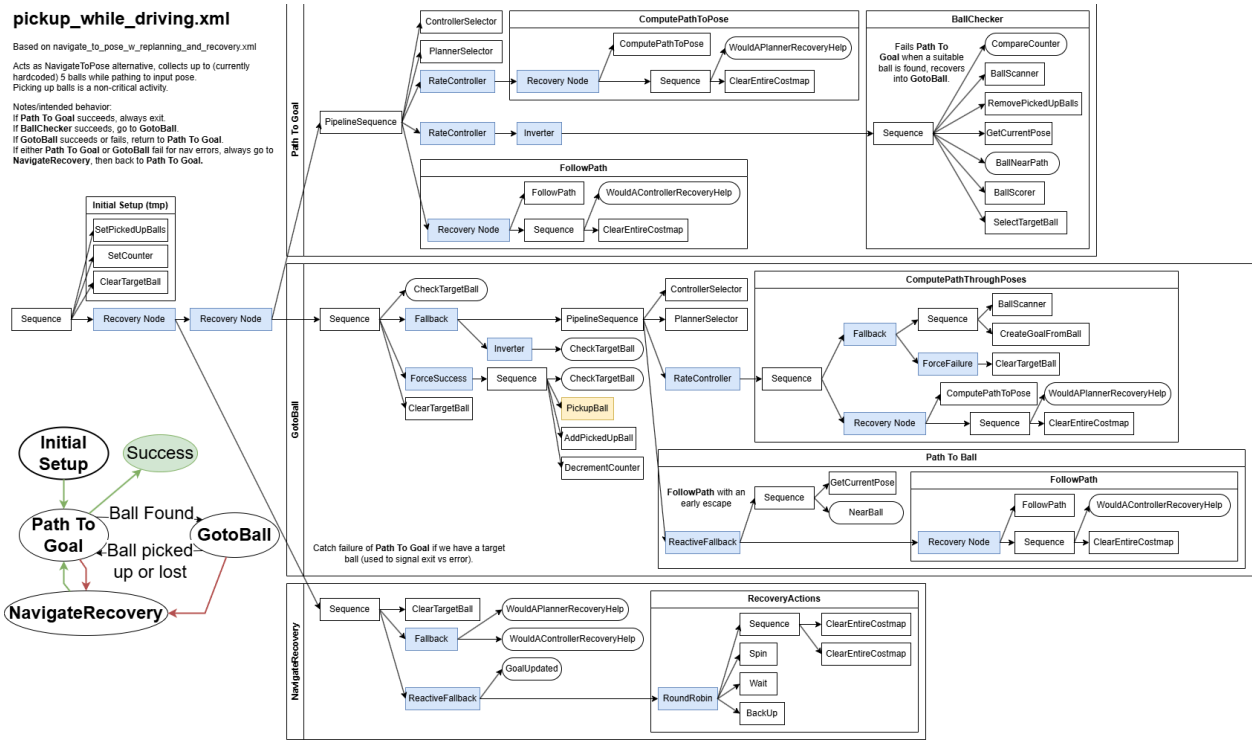

Figure 2: Annotated drawing of the pickup while driving behavior tree

There are two main states in this action: driving ("Path To Goal") and pickup ("GotoBall"). The driving and recovery sections are modified from Nav2's default tree, with an extra check inserted for blocks around the current path. If a block is found, Path To Goal is "failed" and "recovers" into GotoBall, which sees the failure reason being that a block was chosen, and so paths to that block. "Failures" are used for control flow to exit from a tree to re-enter into another one, even if the action has not truly "failed," since a successful exit implies the completion of the action. Once near the block, the block pickup local action is called which returns to Path To Goal. If GotoBall "fails" for anything other than a navigation error it goes back to Path To Goal, and if either encounters a navigation error it goes to recovery which continues returning to Path To Goal until the maximum failure count is exceeded. As it's

implemented entirely through a behavior tree, other actions can utilize the tree to pick up blocks while performing other tasks.

Local actions are simple tasks which run directly using just object detection with direct control over hardware. These simpler actions can be called directly by the Strategy AI or the Action Server, and are used for quick precise actions that can execute independent of localization or the field state. Block pickup is one such local action, as it uses only the YOLO object detection to command the robot to collect up to a requested number of blocks by iteratively centering the closest block one in the front camera view using a PD controller and driving into it.

Simulation

In order to work on software features and troubleshoot effectively, a simulation version of the robot and the VEX field were developed. Gazebo Sim was chosen as the simulation software due to its ease of use and broad compatibility with ROS 2. Initially, the simulated world was intended to only be used for confirming that the robot nodes were correctly configured and eliminating hardware related issues when developing a new feature. Once integrating systems became the main software task, more complex behaviors and features were integrated.

Optimizing for Performance

The high computational cost of simulation and need for higher real time kinematic speed became apparent upon completion of the first version of the environment. Optimizations were made at both the world and model level in order to increase performance. Simulation Description Format (SDF) allows for a visual and collision model to be attached to each link of an SDF file. This means that both visual complexity and faster performance can be obtained with a simplified

collision model that is only used by the physics engine and a detailed model that is only rendered by the visual model.

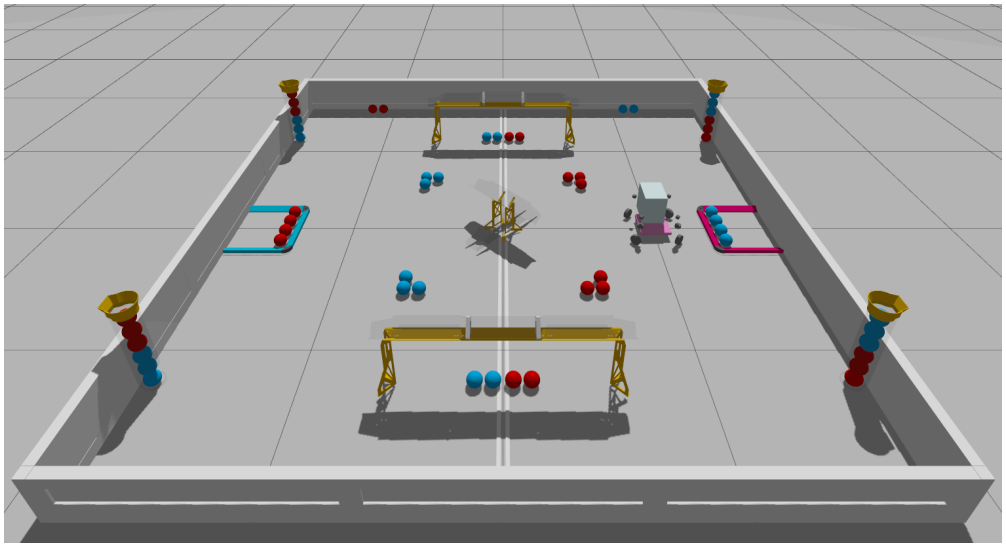

Figure 3: VEX Push Back Field in Gazebo Sim

The initial model of the field that was available on the VEX website has an incredible amount of detail and contains every single component needed for building the structure in real life. While that is great information to have, it is not needed in a simulated environment and there are no repercussions to eliminating those components. Model optimizations were made by importing STEP files into Blender, decimating the mesh in order to reduce triangle count and therefore model complexity, and exporting as a COLLADA (.dae) file. Using this particular file type allowed the model to retain its material properties such as color and can preserve embedded textures if they were present. The collision model consisted of primitive (built in) shapes for the walls and a highly decimated mesh of more complex structures such as loaders and goals. The field ended up consisting of two separate SDF models since certain properties such as transparency can only be set for the entire model and not individual components.

Upon discovering adding around seventy blocks once again drastically slowed down simulation (~3.4% real time with the robot model spawned in), a similar process was applied to the blocks, except both the visual and collision model use only the sphere primitive. After simplifying each model, various world SDF file edits were made, which resulted in much more drastic results. This included setting a limit for the number of collisions that could be simulated and changing the timestep from 10 ms to 100 ms.

Rolling Friction Bug

A commonly encountered and known bug in Gazebo Harmonic is the inability to simulate the friction on rolling objects such as spheres. While in reality the scoring pieces were not spheres, but multifaceted blocks, the built in sphere primitive was used to improve overall simulation performance and boost real time factor calculation speed. This decision became a major problem when implementing pick up and scoring mechanics since after driving one path, and lightly contacting many objects, the majority of the blocks would be pressed against the field walls. Real game elements roll, but not indefinitely. In order to resolve this problem, a Gazebo plugin was developed that applied a force in the opposite direction that was a specific percentage of the current speed, acting as a simple linear friction approximation.

World Interaction

As the year progressed, the robot hardware fell far behind schedule and the world needed to be upgraded to allow for more complex actions and functionality to be tested. The first major update included the addition of a python back end that tracked the poses of each block and

exposed scoring services so that the simulated model of the robot could interact with the world around it.

Services followed a common logical sequence of steps in order to validate that the action requested was actually achievable. We did this by providing two broader services for the robot to call, which would trigger another service to manipulate the entities in the world accordingly. An example of this is the "pickup block" service which would delete a nearby block from the world and update it in the robots inventory. Unexpected behaviors also had to be accounted for such as scoring at the wrong height or attempting to pick up a block when one is not in range.

A great example of dealing with unexpected events is the ScoreBall service. There are checks for both correct pose within a tolerance and scoring at the correct height before the actual scoring of a block is even thought of. If both of those criteria are met, the actual scoring service is called and the blocks are shifted accordingly. If either the incorrect scoring height is chosen or the robot is not within the tolerance of one of the goals, the block that was to be scored is randomly placed on the ground within one field tile (~ 2ft) of the robot, simulating missing the goal.

Robot Design

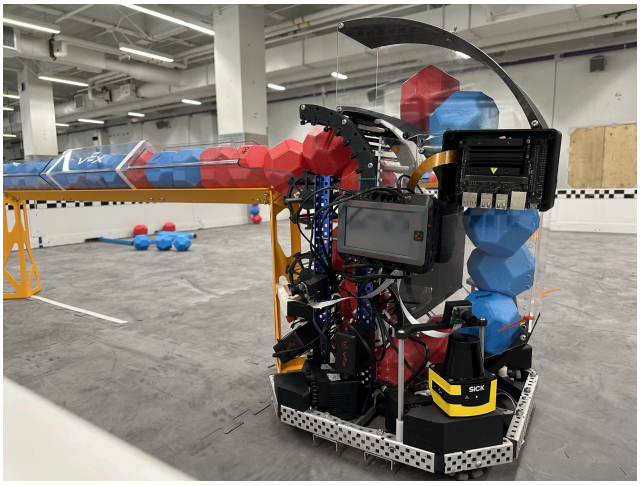

Figure 4: Our robot on the Push Back field

The mechanical design of the robot had to meet the scoring requirements of the “Push Back” game and the constraints of the VEX V5 ecosystem. To compete autonomously the robot must be able to intake blocks from the field, hold multiple blocks at once, and score at each of the three goal heights: bottom-middle, top-middle, and long goal. In addition, the robot must provide sufficient space for non-VEX hardware, including an onboard computer and sensors.

Sensors

For our robot to get information about its environment, we have four cameras mounted on each corner of the robot, along with a fifth camera mounted above the intake. These cameras allow us to detect blocks around the field. The robot also has two SICK LiDARs on each of the back corners of the robot that work together with an IMU mounted in the center of the chassis for localization. We also have a color sensor mounted inside the intake to detect the color of blocks.

Drivetrain

The robot uses an X-drive holonomic drivetrain, consisting of four omni wheels mounted at 45° angles on each corner of the chassis. This allows independent control of translation and rotation, enabling the robot to move in any direction. By decoupling the translation vector from the robot's heading, navigating to a target pose becomes significantly simpler. Unlike differential "tank" drives, which are bound by non-holonomic constraints and require complex spline paths, X-drive allows path-planning algorithms to calculate direct linear trajectories and independent rotational velocities. Mecanum wheels were considered but not used due to the large size of the VEX manufactured meccanum wheels, which would reduce available space for other mechanisms and electronics. The X-drive also provides superior maneuverability and speed compared to mecanum wheels. The mobile base we design and used for this project can be seen below in Figure 5.

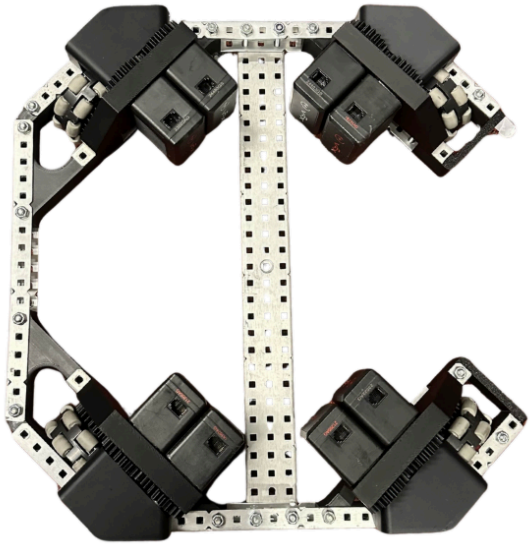

Figure 5: Robot X-Drive Drivetrain

Intake and Scoring

Blocks are controlled through a series of horizontal rubber band rollers stacked into a vertical column. Rubber band rollers were used since they provide the necessary compression while also remaining highly compliant. Furthermore, since the rubber bands can deform almost entirely down to the roller axle, it allowed us to design a significantly more compact roller configuration without the risk of jams. The intake roller at the front of the robot picks up blocks from the field. The indexer roller behind the intake decides whether to send the block to the hopper or up the vertical column. The intake and the roller above it are linked and driven by a single motor, while the indexer and top roller are independently driven. This allows controlled movement of blocks through the system for both storage and scoring.

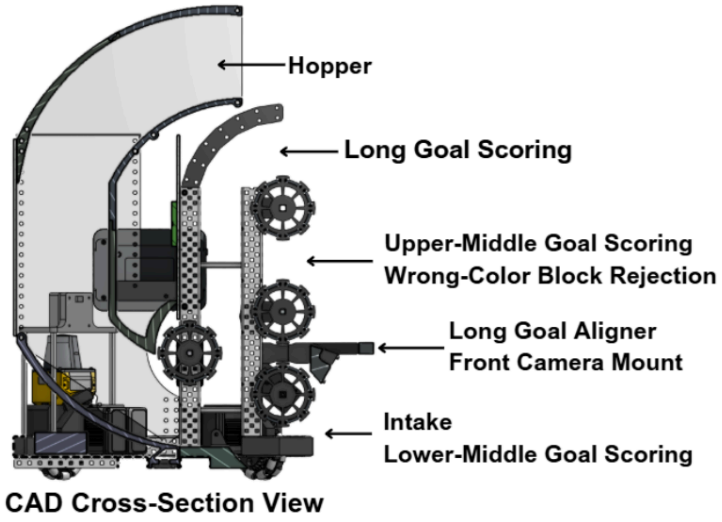

Figure 6: Robot CAD Cross-Section View

The intake roller at the front of the robot also functions as the bottom goal scoring mechanism: by reversing the direction of the roller, it can out-take blocks from the bottom of the robot into the bottom-middle goal. In the middle of the vertical column, blocks can be ejected

from between the top roller and middle roller, this is used for both top-middle goal scoring and rejection of opponent-colored blocks. A color sensor located in the intake detects block color and determines whether the block is kept or ejected.

The top roller is used to score in the long goal by outtaking blocks at the highest point. A fixed aligner is mounted in front of the vertical column for long goal scoring. This triangular hard-stop allows the robot to drive squarely into the goal structure, forcing the robot to be physically aligned with the goal before blocks are scored. This series of horizontal rollers allowed us to use a parallel plate design, containing this whole system to the middle five inches of the robot and leaving plenty of space on the sides of the robot for other hardware.

Block Storage

The internal block storage of the robot underwent many iterative designs to balance capacity and reliability. Our initial design used a large basket-style hopper located at the rear of the robot to maximize the total number of blocks the robot could store. However, testing revealed that the blocks would frequently get jammed in various places and configurations in the hopper. One solution to this funneling issue would be an active agitator mechanism to continuously disturb the blocks and remove jams. The physical footprint of our autonomous computing hardware restricted adding another active mechanism though, and iterations of different passive ramps failed to eliminate the jamming issue. To resolve this without compromising the electronics packaging, we redesigned the storage system into a magazine-style hopper. Instead of one big basket, this new hopper restricts the blocks into a single-file track that curves over the top of the robot. While this slightly reduced the maximum carrying capacity compared to the basket design, it entirely eliminated jamming, The magazine hopper guaranteed block feed

reliability and allows us to easily track and index the exact number of stored blocks in software. Our hopper, as seen along the left and top of the robot in Figure 6 above, can store up to 10 blocks.

Robot Electronics

The electronics hardware for the robot consists of a central NVIDIA Jetson AGX Orin Dev Kit computer ("AGX Orin"), connected to various peripheral sensors and actuators with numerous communications protocols, as can be seen below in Figure 7. The connections between the AGX Orin and peripherals are through either the computer's native ethernet interface over TCP/IP with an onboard network switch allowing multiple connections, or through a custom "Comms Board" PCB to adapt or break-out other protocol connectors.

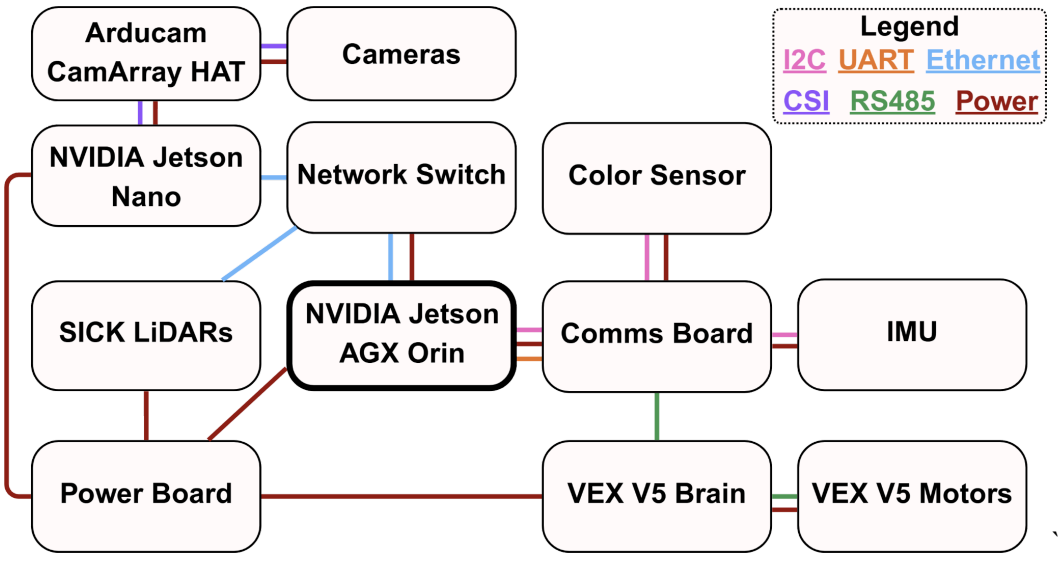

Figure 7: Hardware Schematic Diagram

VEX V5 Brain/Motors

The VEX V5 system communicates primarily through half-duplex RS-485 over its "Smart Port" interface, consisting of a 4P4C RJ-style connector with the RS-485 A/B signals, 12V power and ground. Our setup involves two Smart Port connections between the AGX Orin and the V5 Brain to enable full duplex communications, with one connector being dedicated to the AGX Orin's RX and the other TX. For our purposes, the communications with the brain are treated as just a serial port from both sides, such that we can send arbitrary binary data between the two devices.

We use a simple packet structure for our communications, consisting of a 64-bit magic number to identify the packet type followed by a packed struct. The first packet type is the motor command velocity packet, sent from the AGX Orin to the V5 Brain, and with its struct containing 7 different angular velocity values to set the motors to (one for each of the four wheels and the three intake motors). The commanded velocity values are relayed by the V5 Brain to each motor, and these velocities are maintained until the next command velocity packet is received. These packets are sent periodically by the ROS 2 Control hardware interface, at a frequency of 100hz.

The other main packet types are sent from the V5 Brain to the AGX Orin, and contain readings from each joint's motor encoders. One of the packet types contains the current encoder angular position of each motor, and the other contains the current encoder angular velocity of the motor. These packets are also sent periodically by the V5 Brain, and the data is used by the AGX Orin's software stack to provide the odometry data which contributes towards localization.

Communications Board

We designed a custom printed circuit board (PCB) to facilitate communications between the AGX Orin and the RS-485- and I^2C-based peripherals, as well as to break out the AGX Orin's power rails to provide 3.3V and 5V power to other devices in the system. This PCB is our communications board ("comms board"), which connects to the 40-pin I/O header on the side of the AGX Orin's dev kit carrier board. The comms board sits flat against the side of the AGX Orin's chassis, with the 40-pin header on the bottom side, and all of its connectors breaking out the AGX Orin's I/O and power rail signals on the top side.

The board is two layer FR40 PCB fabricated by JLCPCB, the layout for which can be seen below in Figure 8. At the top center are the 4P4C Smart Port connectors, with the RS-485 transceivers underneath them in the middle. On the top-left corner are the 5V connections, which are 2-pin JST-VH connectors for powering the onboard network switch. Under the 5V power connectors are the three sensor bus I^2C connectors, which are 4-pin JST-PH connectors with clock, data, power (3.3V) and ground pinout. Below those across the whole board is the 40-pin female pin-header connector that goes into the AGX Orin's 40-pin male header. Finally, The bottom edge of the board has debugging headers, and a SPI connector that was left unused. Finally, along the top right is an unused CAN header and transceiver, and another similar I^2C JST-PH connector for interfacing with the power board's fuel gauge I^2C header.

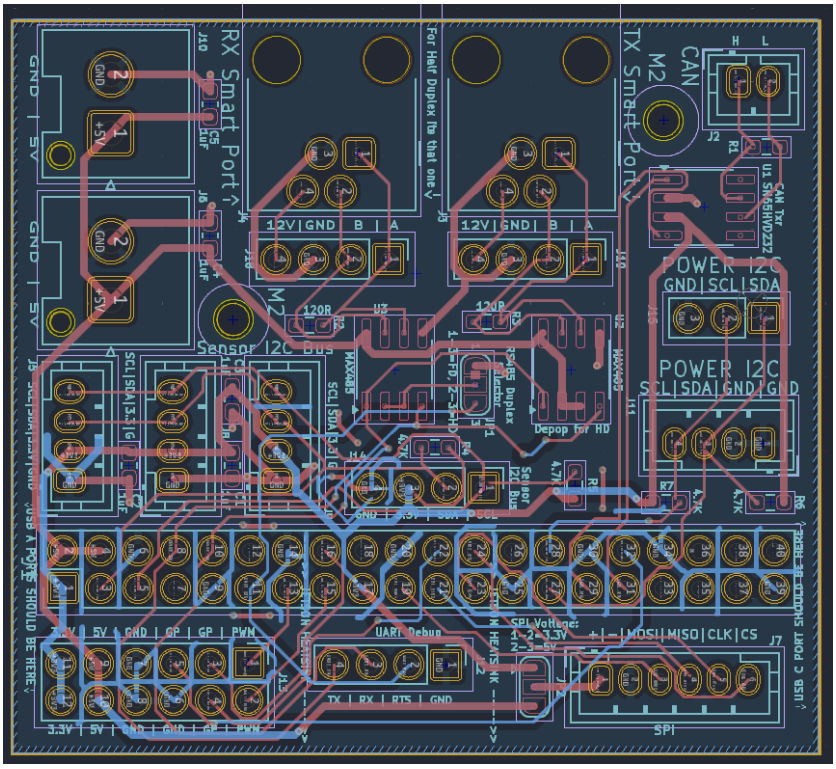

Figure 8: Comms Board PCB Layout

The comms board is critical for facilitating the AGX Orin's V5 Smart Port communication to the V5 Brain, through RS-485. Since RS-485 is a differential-pair based industrial physical layer (PHY) protocol, the AGX Orin cannot output it natively, as it can only directly drive TTL (meaning here, 0 or 3.3V) signals. We use the AGX Orin's UART pins to provide TTL serial to and from the computer, with RS-485 transceiver ICs on the board to encode/decode the signal for transmission to/from the V5 Brain. The comms board has terminals for the same 4P4C RJ-style connectors that the V5 system uses, so we are able to use the same native cables and as the rest of the V5 system to communicate with the brain. As mentioned above, we used two Smart Ports to achieve full duplex communication between the V5 Brain and the AGX Orin, with the goal of simplifying the communications protocol between them. The

comms board also includes a solder jumper switch to allow switching to half-duplex communications.

We decided to use the AGX Orin's power rails to provide 5V and 3.3V power instead of creating our own 3.3V and 5V rails out of the 12V battery onboard for simplicity. The only 5V devices we used are the RS-485 transceivers and network switch, both of which have very low current draw, so the Jetson's external power rails were sufficient for that purpose - if we had any actuators using 5V or 3.3V power, we would have needed to find a different solution. The 3.3V power is distributed through the I^2C JST-PH connectors and used to power the IMU and the color sensor. The power board also has a higher-current-rated 3.3V rail onboard if there was a need for it, but all the 3.3V devices we used were very low draw so we decided not to use that rail externally for simplicity of wiring.

Power Board

To distribute 12V power to the AGX Orin and the two SICK LiDARs, we designed a custom power distribution PCB. The original design intention for this board was to power the V5 Brain and Motors through the same battery as the rest of the system, so the board is designed for high current distribution. The expected power draw of the full system could theoretically be as high as 200W (17A @ 12V), which is a lot of current to be flowing through a small PCB measuring just 40mm on each side, so thermal management was carefully considered when designing.

Outside of just power distribution, the main purpose of the power board is to provide a fuel gauge for the 12V Li-Ion battery that powers the system. The fuel gauge is an ADI MAX17263 IC, which we are using with a 10mΩ sense resistor. We chose this fuel gauge for its

hybrid measurement approach, since the battery is external to the board and would be swapped out and charged outside of the IC's purview. A typical way for fuel gauges to function is with coulomb counters, where they integrate the current through a sense resistor over all time and track every coulomb in and out to get a continuously accurate count of the battery charge level. In our setup though, the batteries have separate chargers in a different circuit, so this approach wouldn't work for us. Instead of a coulomb counter, we used an IC with a ModelGauge m5-EZ core in it, which can quickly estimate the cell's power state from its initial voltage, then further refine the estimation from current flow through the sense resistor.

The layout for the power board can be seen below, in Figure 9. On the top left of the board is the battery input, with the 5W sense resistor directly below it, with large copper pours for its nets and a dense via stitching grid down to the ground plane. All along the top middle is the circuitry for the fuel gauge, with the IC, its 3.3V power regulator and LEDs for displaying the battery status physically. There is also a button in the middle that is used for triggering the LEDs to show the battery status. Next to the button is a large 470uF bulk capacitor, and there are connectors along the bottom and right edge of the board. Three of the connectors are JST-VH connectors, which are used for the AGX Orin and the SICK LiDARs, and the connector on the top right is the Molex Mega-Fit connector used to connect to the V5 Brain. Finally, there is an I^2C connector in the bottom right for the fuel gauge IC to communicate with the AGX Orin, so that the ROS system has visibility into the battery level for display on our visualizer dashboards.

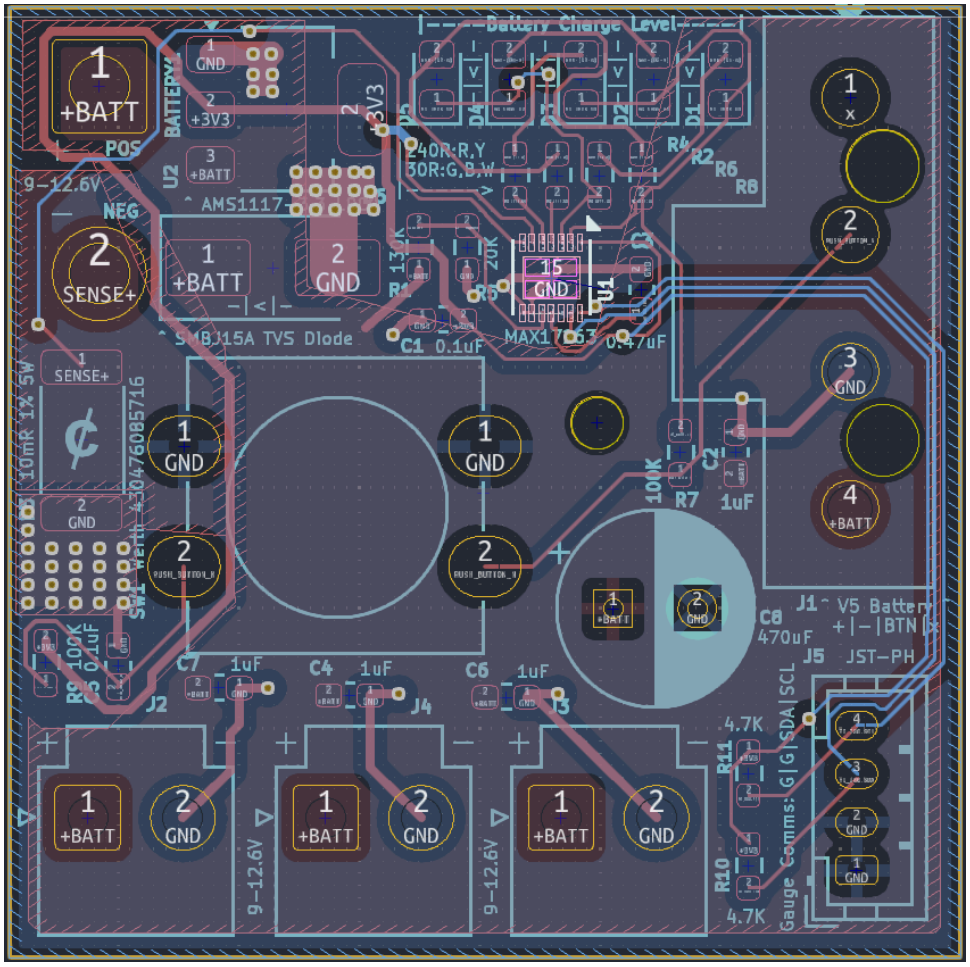

Figure 9: Power Board PCB Layout

To accommodate the high power flow, we designed the power board PCB with maximally large copper pours for the primary rails. Additionally, we used a high-weight 2oz copper PCB to increase the thermal mass of the board, which allows it to carry more current without burning up. The other major concern for thermal management was the sense resistor itself. At 10mΩ with up to 17A of current, we can calculate the resistor's peak power dissipation using $P = I^2R = 2.89\text{W}$, a huge amount for a resistor. We managed to find a high power resistor "metal strip"-style sense resistor, which is rated for 5W continuous dissipation. The copper pours

for the sense resistors are also particularly large, and the ground side of the resistor has a dense via stitching grid to dissipate as much heat as possible to the ground plane.

Cameras

The robot's awareness of the game's field state is driven by cameras feeding into a vision model. We decided to use a fusion of multiple cameras to cover a 360° field-of-view to maximize the robot's situational awareness of the whole field at all times. Given our X-drive mobile base design, the most obvious camera placement setup was to have 4 cameras placed on the corners of the robot, just above each wheel. With this placement, the cameras would be facing symmetrically outwards from the center, and would be easier to design the rest of the robot around compared to the sides.

The choice of camera systems for our robot started with a few requirements: the cameras had to be global shutter to give a clear image while moving, very wide angle to maximize what the vision model could ascertain about the world state, in budget, and we had to be able to connect four of them to the Jetson with, ideally, minimal hassle. Initially, we considered using either the Intel Realsense D435 or an AR0234-based USB global shutter camera since they were cheap, easily approachable, and we had them on hand. After initial testing, we determined that the Realsense's 70 degree horizontal field of view was too small for full field object detection. The wider 120 degree field of the USB global shutter cameras allowed us to see the entire field, with overlap, through only four cameras. After determining that the global shutter camera was a good fit we ordered 4 for the robot.

Over more than a month of attempting to debug our 4-camera setup on the Jetson, we eventually determined that these cameras were a dead-end and abandoned them. The problem

was that they used USB 2.0, for which all USB ports on the Jetson routed to the same legacy USB 2.0 controller in hardware. Attempting to read from more than two of these cameras at the same time would fully saturate the USB bus and no more data could be read. We tried to perform tricks with the driver configuration, by using V4L2's camera quirk settings, and by changing the USB Alt mode of the cameras to use less bandwidth. Eventually though, we decided to try a different path towards multi-camera vision after making no headway on solving the problem.

After the failure of the USB cameras, we decided to use a solution designed more specifically for our use-case, and ended up turning to ArduCam, a commonly used camera vendor for small robotics and other embedded electronics projects. In particular, we settled on their CamArray HAT product in conjunction with a quadrascopic global shutter OV9782 camera array. Between these cameras having global shutter sensors and the CamArray HAT offering shutter-time synchronization, we were confident that this would provide us with even better in-movement performance than our previous solution. The CamArray HAT connects to its host computer through a single CSI connector, and the product listing for this solution included in the title, "for... NVIDIA Jetson AGX Orin," so we expected a simple plug-and-play solution to our camera struggles.

The Arducam CamArray HAT-based solution also did not turn out to be an easy answer to the camera setup for our particular usecase. Despite the product listing calling out the Jetson AGX Orin explicitly, it turned out that the required CSI connector is not present on the official Jetson Dev Kit carrier board we were using, and required another adapter board to use. We were already over budget from the cameras, and were not able to get these working on our AGX Orin.

Our last ditch solution to the camera problem, which we were able to get working, is to use the CamArray HAT on a separate Jetson Nano that a team member had available. The Jetson

Nano does have a CSI connector, and after some trouble with ArduCam's very fickle drivers, was able to reliably read all four cameras correctly through the CamArray HAT. The Jetson Nano is used to simply just read the four cameras through its CSI port, and forward the frames over Ethernet to be read by the AGX Orin on the other side. Since we already had a gigabit ethernet switch on the robot for communicating with the LiDARs, this was doable with the extra addition only of the Jetson Nano itself.

The eventual solution of using the Jetson Nano as an intermediary to forward the camera feeds is clearly not ideal - it adds unnecessary complexity, failure points and power draw to the system, as well as increasing frame latency and adding a throughput bottleneck to the system. While this solution is not an ideal one and has many flaws, it was the only consistently working solution and allowed us to make further progress in terms of hardware integration.

I^2C Drivers

We wrote our own ROS 2 drivers and driver adapters for most of the I^2C devices we used. In particular, we had to create ROS 2 drivers for the STM LSM6DSV1X IMU, the ADI MAX17263 fuel gauge, and the APDS9960 color sensor. The LSM6 IMU ROS 2 driver we created is based on the existing C driver provided by STM, with an adapter layer to configure the chip to our needs, poll the I^2C interface to get readings, convert the data-types to ROS-friendly ones and continuously publish the readings on a ROS topic.

The ADI fuel gauge driver had to be written from scratch, as no existing C driver was available. Thankfully, this is a relatively simple IC and the driver was easy to write. We added a ROS 2 adapter on top of the driver once again, which configures the chip as needed and polls to get readings, convert datatypes and publish.

The APDS9960 color sensor driver was pre-existing though third-party and just required a ROS 2 adapter layer. Initially, we ran into a lot of trouble getting the readings on this chip to make any sense. After some investigation, we found very fundamental bugs in the driver source code, which, puzzlingly, made the readings for the primary purpose of the sensor always completely incorrect, and upstreamed the bug fixes. The ROS 2 adapter layer for this sensor was once again simple and much like the others.

Computer Vision - Image Processing

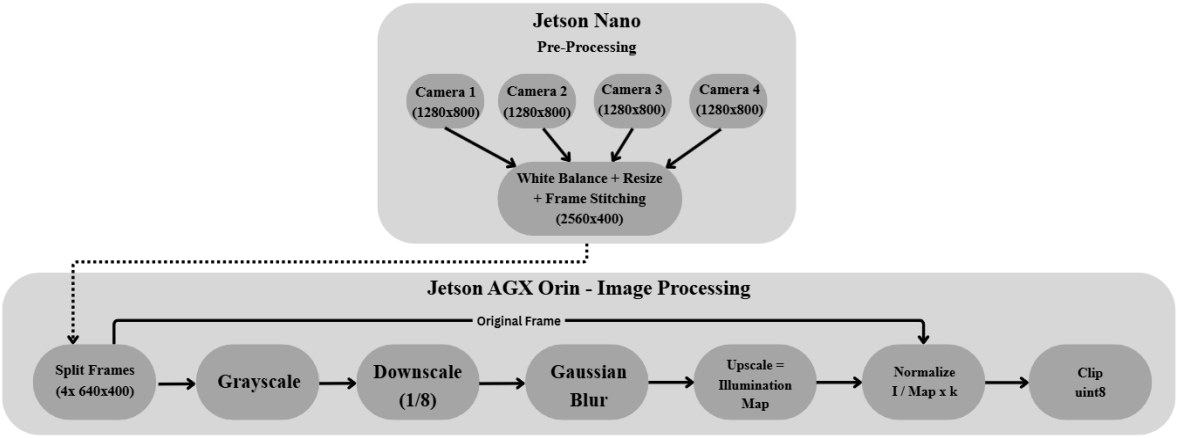

Figure 10: Camera Processing Pipeline

Figure 11: White Balance Correction Comparison (Top: Before, Bottom: After)

The Jetson Nano reads from the Arducam CamarrayHAT, capturing the four synchronized camera streams from each corner. Each camera frame is resized from 1280x800 to 640x400 and stitched horizontally into one 2560x400 strip. The raw images out of the cameras are very dark and have a strong yellow tint, which makes object detection difficult. To address this, the images are color balanced on the Jetson Nano before being streamed through a TCP socket over ethernet to the primary Jetson AGX Orin.

Illumination Correction

Even after the white balance correction, the images still had a very strong vignette effect, which can be seen in the bright radial hotspot in the center of each camera image in Figure 11. To resolve this, we designed a real-time illumination correction pipeline based on Retinex theory (Land, 1977). Retinex theory models human color perception by modeling an observed image $I(x, y)$ as the product of the ambient illumination $L(x, y)$ and the true object reflectance $R(x, y)$:

$$I(x, y) = L(x, y) \cdot R(x, y)$$

The goal of illumination correction with this model is to isolate the true reflectance $R(x, y)$ and make the illumination $L(x, y)$ even across the image to reduce the bright hotspot. We achieved this through a six-step process using OpenCV:

1. **Grayscale Conversion:** The original BGR frame is converted to a single-channel grayscale image to isolate the luminosity data
2. **Downscaling:** To minimize the computational expense, and to further equalize the luminance, the image is downscaled to 1/8 its original resolution
3. **Illumination Estimation:** A Gaussian blur is applied to approximate the illumination field. The blur removes fine textures, leaving only the smooth illumination changes across the image.

$$G(x,y) = \frac{1}{2\pi\sigma^2} exp(-\frac{x^2+y^2}{2\sigma^2})$$

$$L(x,y) = I_{downscaled}(x,y) \cdot G(x,y)$$

4. **Illumination Map:** The blurred illumination estimate $L(x,y)$ is resized back to the original 2560x400 resolution. To prevent mathematical operation errors in the next step, all pixel values in this map are clipped to a minimum value of 1.
5. **Reflectance Recovery and Balancing:** The original BGR frame is converted to a floating-point array. The original frame is then divided by the illumination map to recover the object reflectance. The reflectance is then scaled by a target baseline brightness ($\beta = 120$).

$$I_{corrected}(x,y) = (\frac{I_{original}(x,y)}{L(x,y)}) \cdot \beta$$

6. **Clipping and Re-casting:** The balanced floating-point array is then clipped back to the uint8 range of [0,255] to produce the final corrected image.

This Retinex-based pipeline flattens and equalizes the luminance across the entire image effectively removing the radial bright hotspot. The whole pipeline is visualized step-by-step in Figure 12 below.

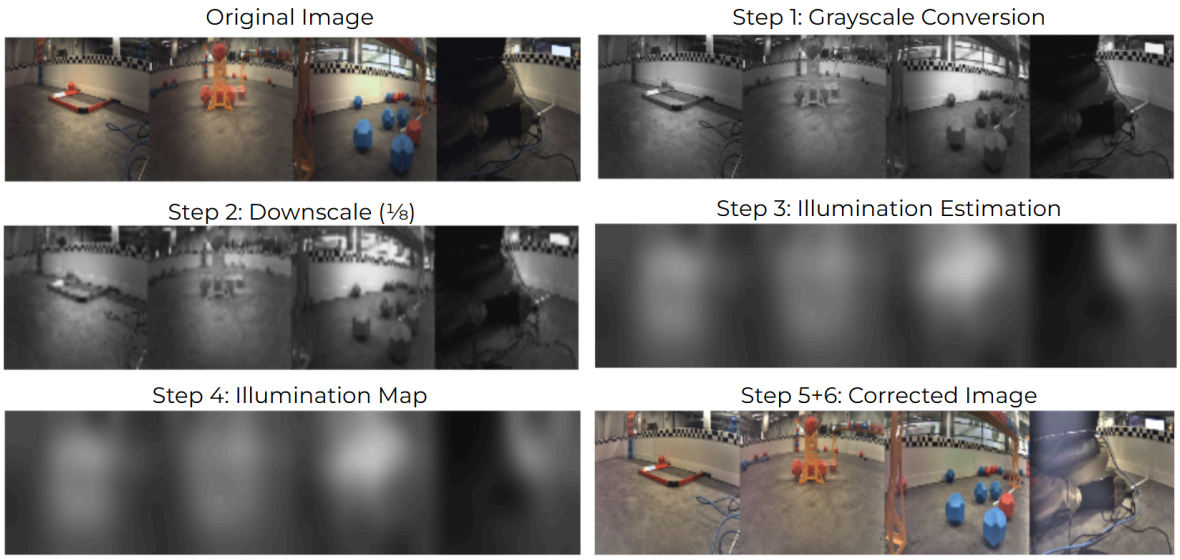

Figure 12: Image Processing Pipeline Steps

Computer Vision - YOLO

We trained a YOLO v11 model with the goal of detecting game objects, specifically the red and blue blocks. This model detects and provides bounding boxes for each game object in the camera frame, allowing us to estimate their robot relative positions in real time. This YOLO model was intended for use on the video feed from the front-facing camera to allow the robot's local actions to precisely track the game objects and, for instance, help center a block in the robot's intake.

YOLO Block Detection Model

YOLO v11 models come in a variety of sizes: nano, small, medium, large, and extra large, ranging from 2.6 million parameters for yolov11n (nano) to 56 million parameters for yolov11x (extra large). When choosing a model size, it was important to have frequent detections while maintaining accuracy. This led us to choose the yolov11m (medium) model architecture. In

COCO accuracy benchmarks, yolo11m has a MAP@50-95 of 51.5. This is a 10% improvement over the smaller yolov11s and is only 5.9% behind the significantly larger yolov11x architecture (Ultralytics, n.d.). On our planned hardware platform at the time, the Jetson Orin Nano, yolo11m was measured at a ~30ms inference time allowing for 30hz object detection (Vina, 2025). While this is slower than smaller models like yolo11s which has approximately half the inference time, a 30hz refresh rate is sufficiently fast for real-time navigation during gameplay.

Roboflow Dataset

To train the YOLO model, we found an open source dataset of 1352 images hosted on the Roboflow image labeling platform. These images were relabeled using the platform to ensure bounding boxes were drawn around all objects and all object classifications were accurate. In addition, we supplemented the open source dataset with additional images taken on the Push Back field. Before training, we augmented the dataset using roboflow's data augmentation tools to artificially create a wider range of lighting conditions, exposure, and camera orientation in the training data. This is intended to help the model better adapt to a wide variety of cameras and lighting conditions. For each image in the original dataset, three were added to the augmented dataset. The brightness, exposure, orientation, and rotation of the augmented images were randomly modified. After applying the augmentations, we trained the final yolov11m model on a final dataset of 3284 images using the Turing HPC cluster.

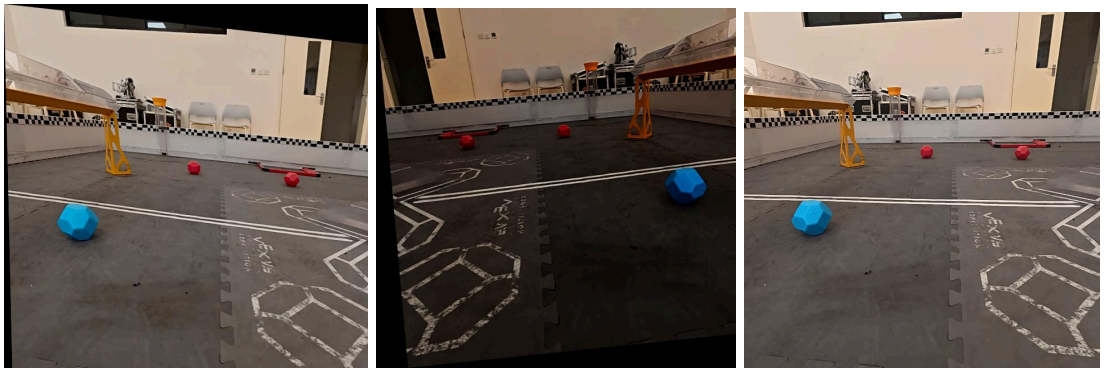

Figure 13: Example of Augmented Data

YOLO ROS 2 Node

To deploy the YOLO model on the robot, we created a ROS 2 node to read camera frames at 30hz from a global shutter camera, process the frames with the yolo model to detect objects in the frames, and publish the results on a topic to be used by actions or other nodes. This node has launch parameters allowing us to change the file path to the model, confidence threshold for detections, and output topics for an annotated image and detections. We chose to read directly from the USB camera rather than through an intermediate node to publish camera frames in order to optimize for speed and minimize latency.

We used the ROS 2 standard vision_msgs/Detection2DArray message type and constantly publish detections as camera frames are processed. Each Detection2D has a BoundingBox2D with corners extracted directly from the YOLO model output. In order to enable the robot to track individual objects over multiple frames, the Hypothesis2D's class_id is modified to include a unique ID in addition to the name of the class from the model output. For example, a "red" detection with id 12 would have a class_id of "red_ID12". This class, combined with the confidence from the YOLO model output makes up the full detection output. As the YOLO model only detects objects in image space, the ObjectHypothesis' pose is left empty. To

enable efficient debugging and tuning of the model and confidence threshold, an annotated image with bounding boxes, classifications, and confidences is also published in addition to the detections.

Computer Vision - Unified Game State Vision Model

Given the camera feeds from our 4 onboard cameras, the goal of the unified game state vision model is to localize blocks and opponents on the field. We created a custom AI vision model architecture and dataset to accomplish this and create an occupancy map of where the blocks are in 2D space on the field.

Dataset

All experiments and training runs used a dataset of 200,000 synthetic samples created with Blender (Blender Online Community, 2025). We started by taking the official CAD model of the game field, and simplifying its geometry from 30 million triangles down to 150 thousand for faster rendering and lower memory usage. Blender's default rendering settings such as Noise Threshold, Max-Min sample were also reduced to maximize render speed, giving very little observable degradation in quality and enabling ~10x faster rendering. Each sample consisted of a rendered view from each of the 4 corner cameras, and a JSON file containing metadata about each block, such as its global coordinates, colour, bounding box and placement on the field. The dataset was generated on 8x NVIDIA L40S GPUs within 16 hours. See Figure 14, 15, 16.

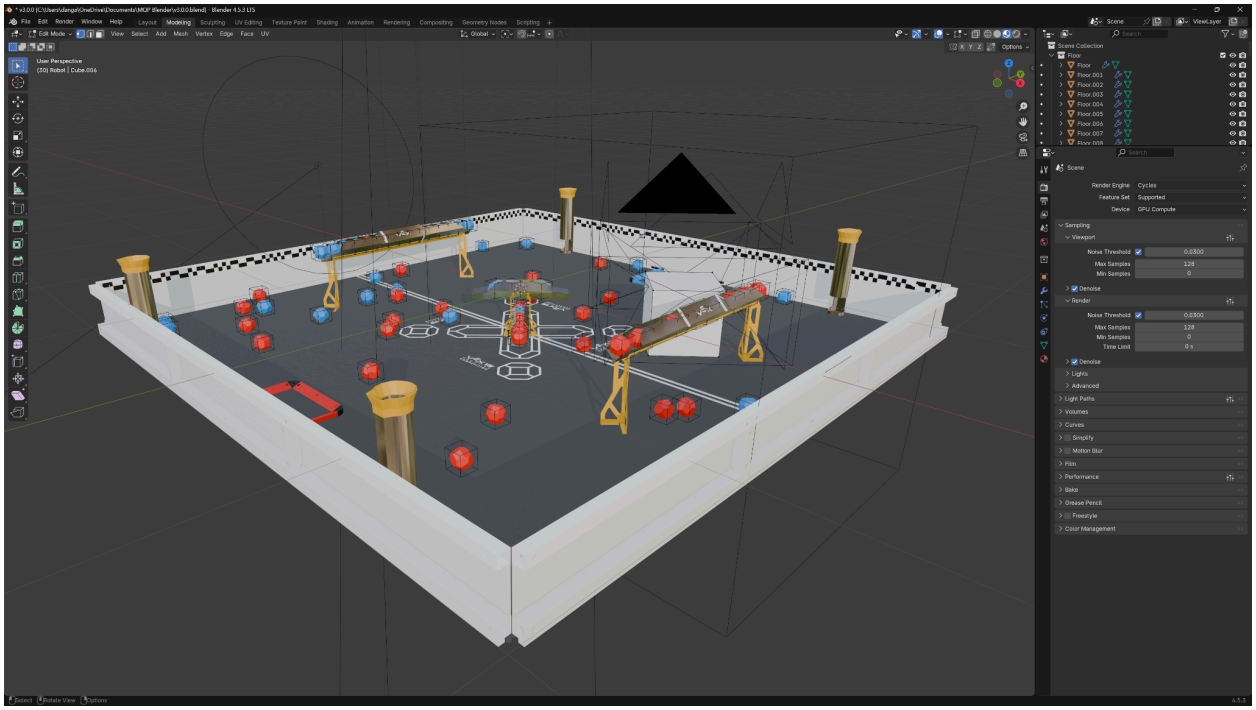

Figure 14: Blender Viewport

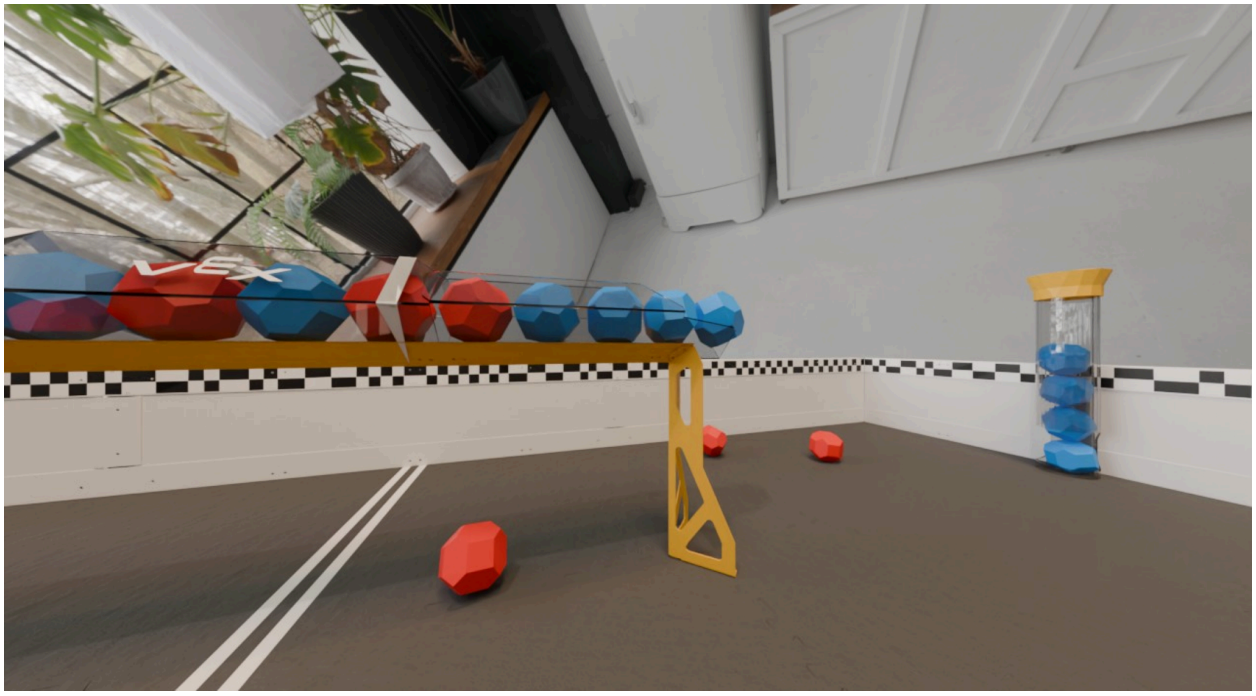

Figure 15: Single camera render

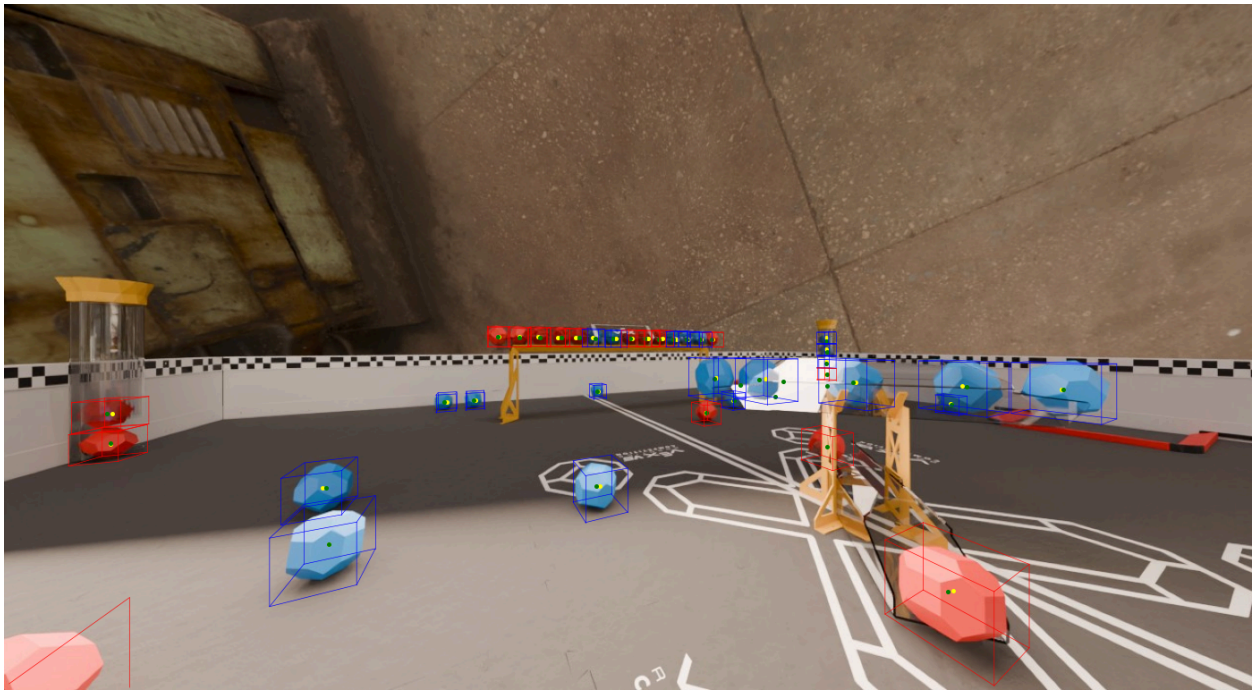

Figure 16: Example of glare in render.

The renders were generated with a resolution of 1280x704. This resolution was chosen for the general goal that objects should roughly occupy a minimum 32x32 pixel grid in the render to allow for robust detection. We also had to consider the trade-off between render time and quality, as well as inference computation required for the model.

It is important that during inference, the model can be robust to different backgrounds. To achieve this, a random HDRI background image was added to Blender, placed at a random rotation. The goal of this is to have a background-agnostic model that will be able to work across any room we put it in. In total, 25 unique HDRIs were used for data generation, chosen randomly for each sample. Similarly, we generated random lighting for the renders to make sure the model could be robust to different lighting environments. The block's location and numbers were spawned randomly to ensure diversity as well.

Object Detection

Throughout the development of our vision model, we experimented with 2 fundamentally different approaches to object detection to determine which would perform best for our needs:

1. **Discrete occupancy grid prediction**: By rasterising the VEX field into an NxN grid, we can train a model to predict the occupancy of each cell. An occupied cell means a block is present. We chose N = 48, as this size would make each cell containing at max 1 block and avoid the model having to deal with multi-cell prediction. In practice, this task simplifies to a classification problem with 3 classes for each NxN grid cell: blue block, red block, and empty.
2. **3D coordinate prediction**: In this approach, we trained a model to directly predict the 3D coordinate of each block along with their colours. This task can be formulated as multi-object detection in 3D (similar to standard object detection in a 2D image).

Discrete Occupancy Prediction

Dataset Preprocessing

To train a model to perform discrete occupancy prediction, we needed a discrete occupancy dataset. Notably, our original data samples included both the camera images and each block's XYZ coordinates. We applied a continuous to grid transformation by assigning each block to a corresponding cell, where each cell represents a 3" square in the field. Since the occupancy grid is a top-down Birds-Eye-View (BEV) grid, for each loader, we duplicate each cell 6 times (loaders are vertical and can hold 6 blocks each), turning the output space from 2D to 3D. To summarize, the field is discretized into a 48x48 grid, where each long goal is

represented at 15 cells, each center goal is represented as 7 cells and each loader is represented as 6 cells. The entire output space can be modeled as a 6 x 48 x 48 grid (13,824 cells).

Predicting occupancy of this 3D grid is very inefficient and memory heavy since the grid itself is extremely sparse, with at most 88 cells occupied out of 13,824 (~0.006%). We can instead reformulate the problem as a class-specific grid occupancy prediction to reduce complexity. More specifically, we can split the global output grid into 4 smaller grids:

- Grid 1 - Ground: A 48 x 48 grid representing occupancy of the ground.
- Grid 2 - Long Goals: A 15 x 2 grid representing occupancy of the 2 Long Goals.
- Grid 3 - Center Goals: A 7 x 2 grid representing occupancy of the 2 Center Goals.
- Grid 4: Loaders: A 6 x 4 grid representing occupancy of the 4 loaders.

With this new formulation, the output space is now 2,372: an ~82% reduction.

Neural Network Overview

The specifics of our neural network for this prediction paradigm evolved throughout the project, but the core components and input, output and loss specifications did not change (Figure 17). Our architecture takes 4 RGB images from the cameras as input, and outputs the classifications of the 4 grids mentioned above. For each cell in the grid, the neural network predicts if the cell is either blue, red, or null (empty).

Our architecture contains 3 core components. First, following common practice in 2D computer vision, the images are passed through a feature extractor. This is commonly done using a ResNet (He, 2015) backbone, optionally enhanced by a Feature Pyramid Network (Lin, 2017) for multi-resolution feature fusion. Second, a Transformer (Vaswani, 2023) aggregates spatial information from image features to position queries using cross-attention. Third, a detection head

(in our case 4 heads, 1 for each grid) computes the probability of occupancy for each cell. Finally, we evaluate cross-entropy loss to train the neural network.

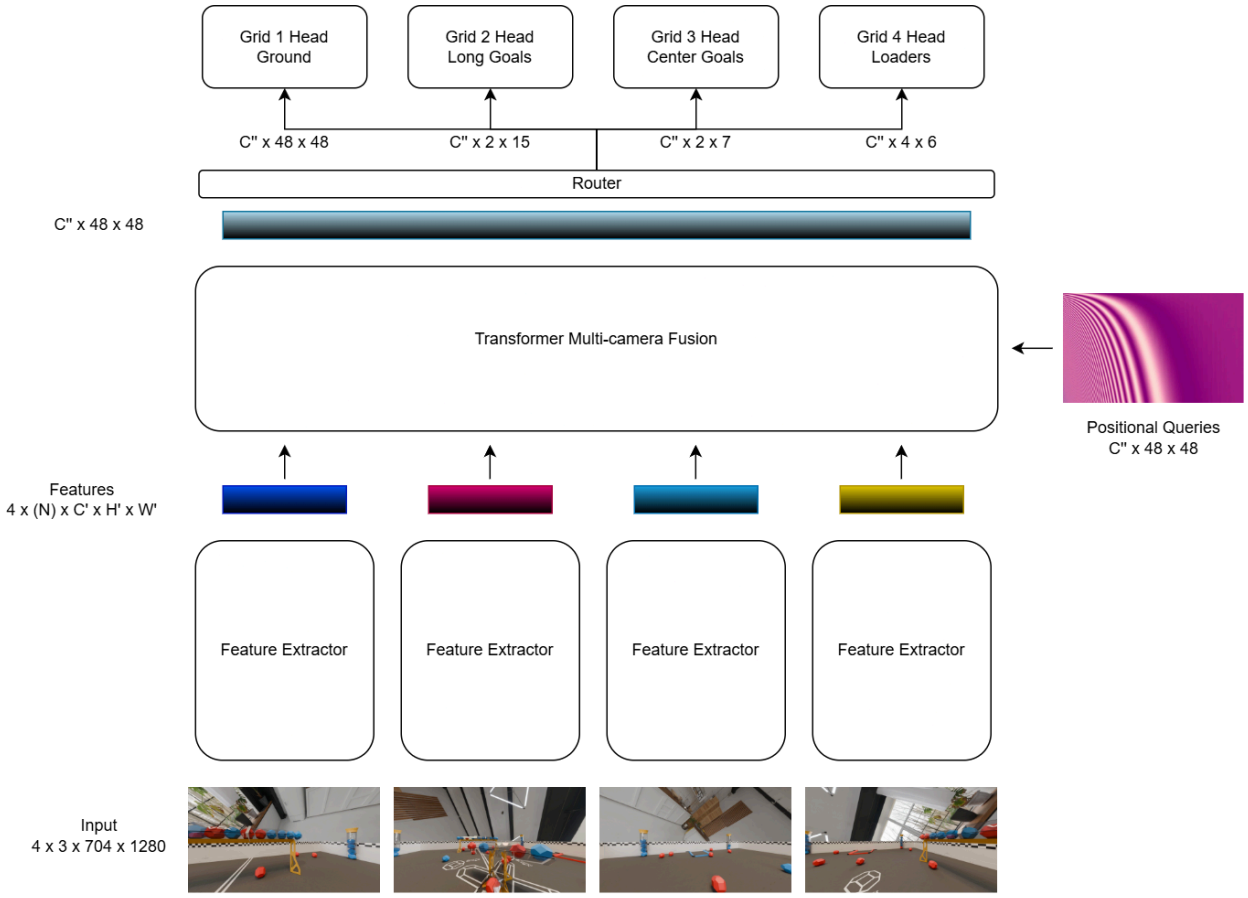

Figure 17: General architecture of discrete occupancy prediction. Each image is processed by a feature extractor backbone. A Transformer then performs feature fusion along with aggregating image information to positional queries. A router then routes each queries corresponding to its position in the output grid to its designated head.

Feature Extractor

We start with a set of images $I = \{im_1, \ldots, im_4\} \subset \mathbb{R}^{3 \times H_{im} \times W_{im}}$ from the 4 onboard cameras. These images are encoded with a ResNet and a FPN, Figure 18, into 4 sets of features:

$\mathbb{F}_1, \mathbb{F}_2, \mathbb{F}_3, \mathbb{F}_4$. Each set $\mathbb{F}_k = \{f_{k1}, \ldots, f_{k4}\} \subset \mathbb{R}^{C' \times H' \times W'}$ corresponds to a level of features for the 4 images.

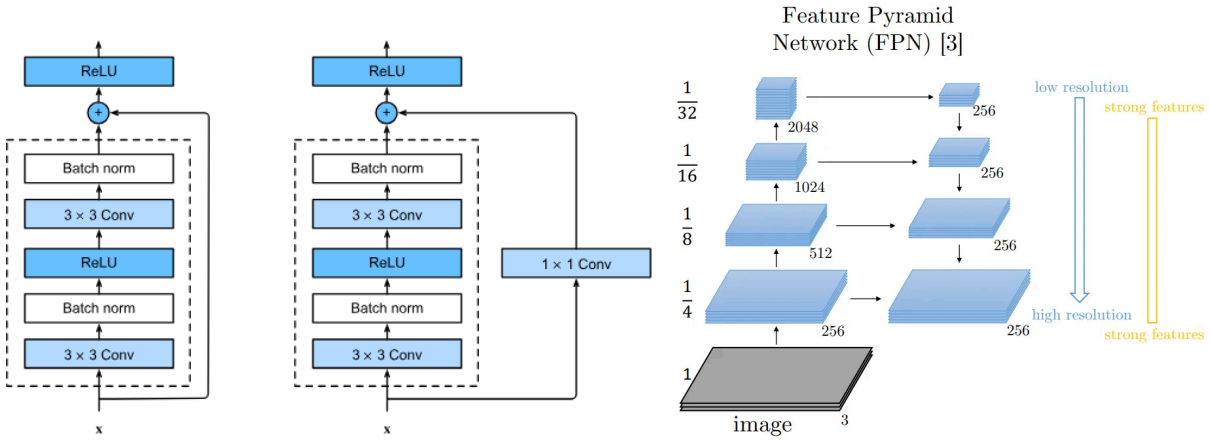

Figure 18: *Left*: Illustration of ResNet, a class of CNN with residual layers, allowing better gradient flow leading to more stable training. *Right*: Illustration of FPN. Lower level features get aggregated to higher level, leading to a stronger image representation.

This multi-scale feature formulation is common in most feature extractor backbones (some might use BiFPN instead of FPN). Our implementation of feature extractor differs in 3 ways to the standard conventions developed in the autonomous vehicle field:

1. **ResNet only:** While FPNs provide stronger features, they are computationally heavy. We experimented with and without using FPN and did not see any performance drop. A possible reason why is that compared to autonomous driving settings, our dataset contains simple features with only 1 type of object (blocks), so high capacity is not needed.
2. **Shared ResNet across images**: Instead of training individual ResNets for each camera input, we share one set of parameters across all 4, saving memory and computation. Since

our robot is on a symmetrical X-drive and all cameras are mounted in an identical manner, the image view looks very similar between them. Therefore we should not need to train separate ResNets like autonomous vehicles do.

3. **Initialised from scratch**: Most autonomous driving research papers use a pre-trained ResNet on Imagenet or nuScenes in order to avoid learning feature extraction. This can lower training time. However in our case, VEX fields and VEX blocks are not common, hence we decide to train ours from scratch.

Our implementation of ResNet follows ResNet18 ~12M parameters. We also experimented with other ResNet variations such as ResNet-X and ConvNet but saw no marginal improvements. We also experimented with different ResNet sizes such as ResNet34 and ResNet50, ResNet18 trained the fastest with similar performance.

Camera Fusion

After extracting features from each camera view, we fuse all image features together before performing object detection. This early fusion strategy allows the model to reason jointly across views.

In contrast, a widely used alternative is late fusion, where each camera view is processed independently by a separate (or common) detector. The resulting predictions (bounding boxes, scores, or occupancy maps) are merged afterward using techniques such as Non-Maximum Suppression (NMS) or Weighted Fusion. While simple and modular, late fusion fails to exploit important cross-view information during feature extraction. This often leads to missed detections, low-confidence predictions, or inconsistent 3D localisation.

For these reasons, we adopt Transformer-based fusion in our model. By integrating multi-camera features internally within the network, it effectively captures cross-view correspondences, providing richer and more reliable 3D positional information for each block (Li, 2022).

Positional Queries

We represent the BEV plane as a fixed grid of learnable positional queries. Specifically, we predefine a group of grid-shaped parameters $Q \in \mathbb{R}^{C'' \times 48 \times 48}$ as positional queries where 48x48 is the spatial shape of the BEV plane.

Each query $Q_p \in R^{1 \times C''}$ located at $p = (x, y)$ is responsible for aggregating and representing the features of the corresponding region in the BEV space, see Figure 19. These queries act as positional anchors that enable the Transformer to explicitly reason about spatial locations in the 3D world from a top-down perspective.

Following common practice in ViT (Dosovitskiy, 2020) we used 2D sinusoidal positional encoding instead of learnable embeddings to explicitly inject absolute spatial information. The 2D sinusoidal encoding is formulated as:

$$[PE_{(x,y)} = \left[\sin\left(\frac{x}{10000^{2i/C''}}\right), \cos\left(\frac{x}{10000^{2i/C''}}\right), \sin\left(\frac{y}{10000^{2i/C''}}\right), \cos\left(\frac{y}{10000^{2i/C''}}\right) \right]_{i=0}^{C''/2-1}]$$

Where x, y are normalised grid coordinates, and i indexes the feature channels.

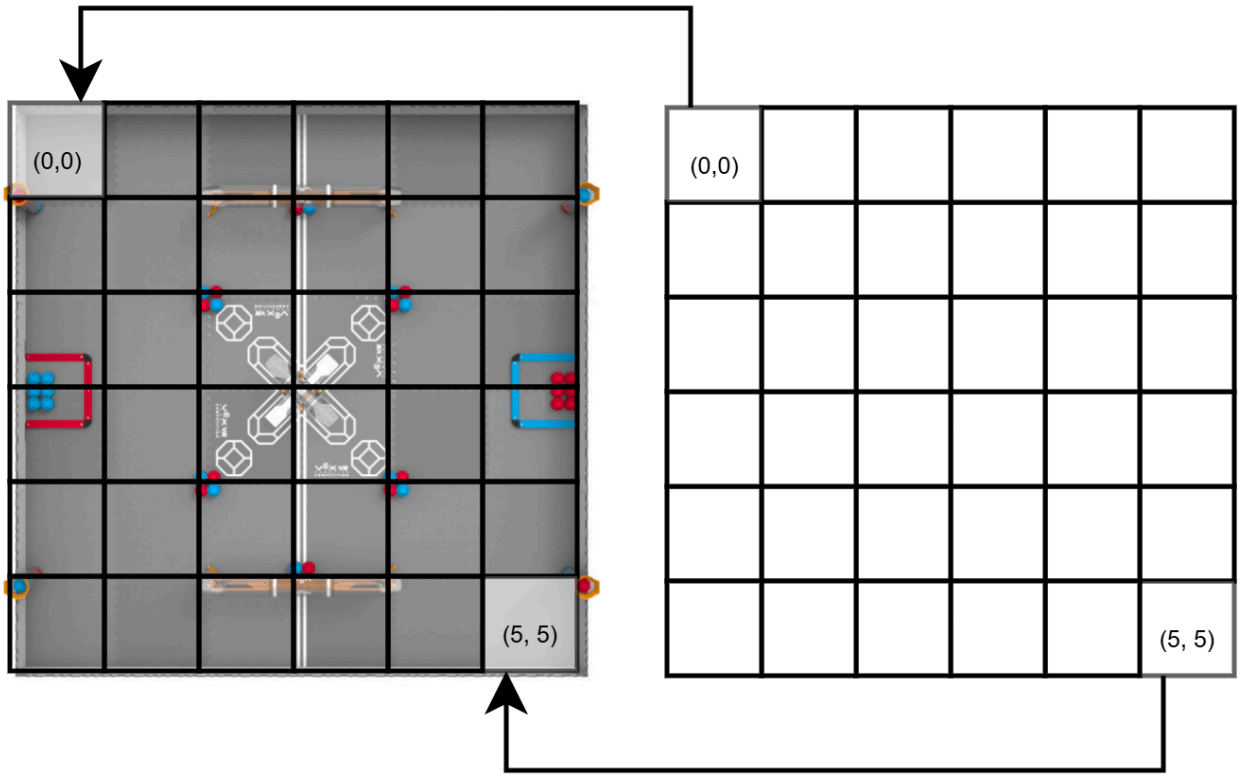

Figure 19: BEV plane illustration (*right*). Each cell in the BEV plane is encoded as a sinusoidal positional embedding and represents the corresponding cell at the same position in the output plane (*left*).

Transformer Encoder

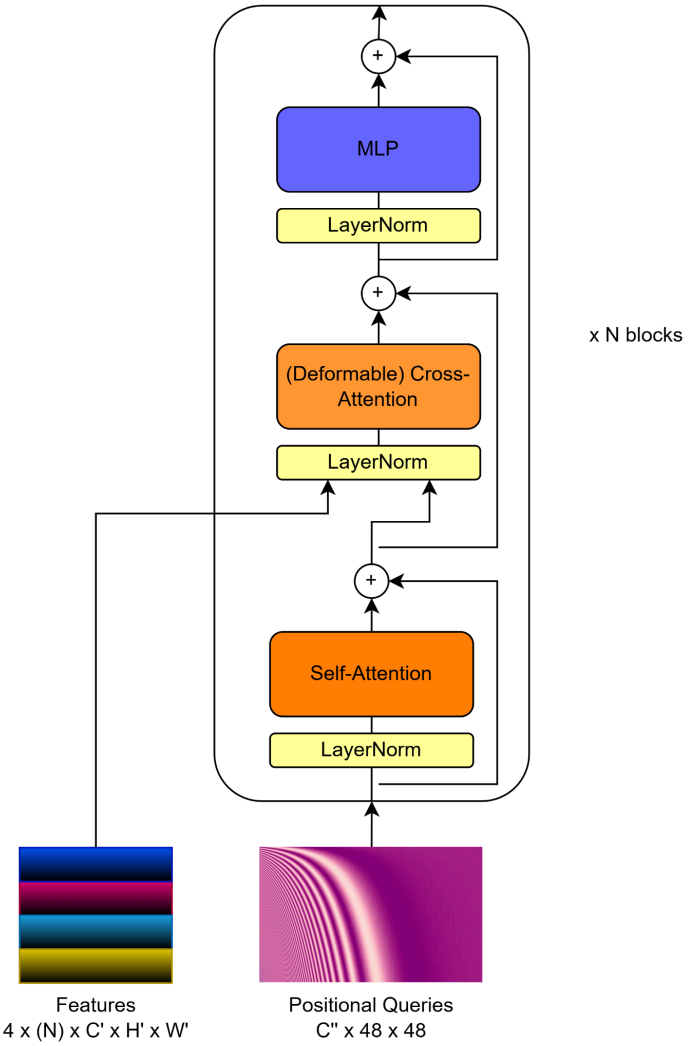

Figure 20: Transformer Encoder Block. Applies Cross-Attention on image features and positional queries.

We use a Transformer to pool information from image features into positional queries, enriching them with spatial information for downstream detection. Specifically we use an encoder block with Self-Attention (SA) over positional queries and Cross-Attention (CA) over images features and positional queries, Figure 20. Below is the formulation of SA and CA where Q, K, V are linear projections of either positional queries or image features:

$$SA(Q, K, V) = softmax(\frac{QK^T}{\sqrt{C''}})V$$

$$CA(Q_{pq}, K_{img}, V_{img}) = softmax(\frac{Q_{pq}K_{img}^T}{\sqrt{C''}})V_{img}$$

Intuitively, CA says: “Given a position in the output plane, where should I look in the image space to better inform detection”.

With the formulation above, training was unstable, and both train and validation loss diverged (Figure 21). We attribute this to 2 issues:

1. **Imbalanced labels**: Even with the reformulated grid of 2,372 cells, the ratio of occupied cells to unoccupied is still 22:571 (~99.7% of all cells are unoccupied).
2. **Cross-Attention cannot learn**: With the ResNet18 feature extractor, the output resolution for each image feature is 22x40. The Transformer must find relationships between 3,520 image tokens and 2,304 positional query tokens, a difficult task.

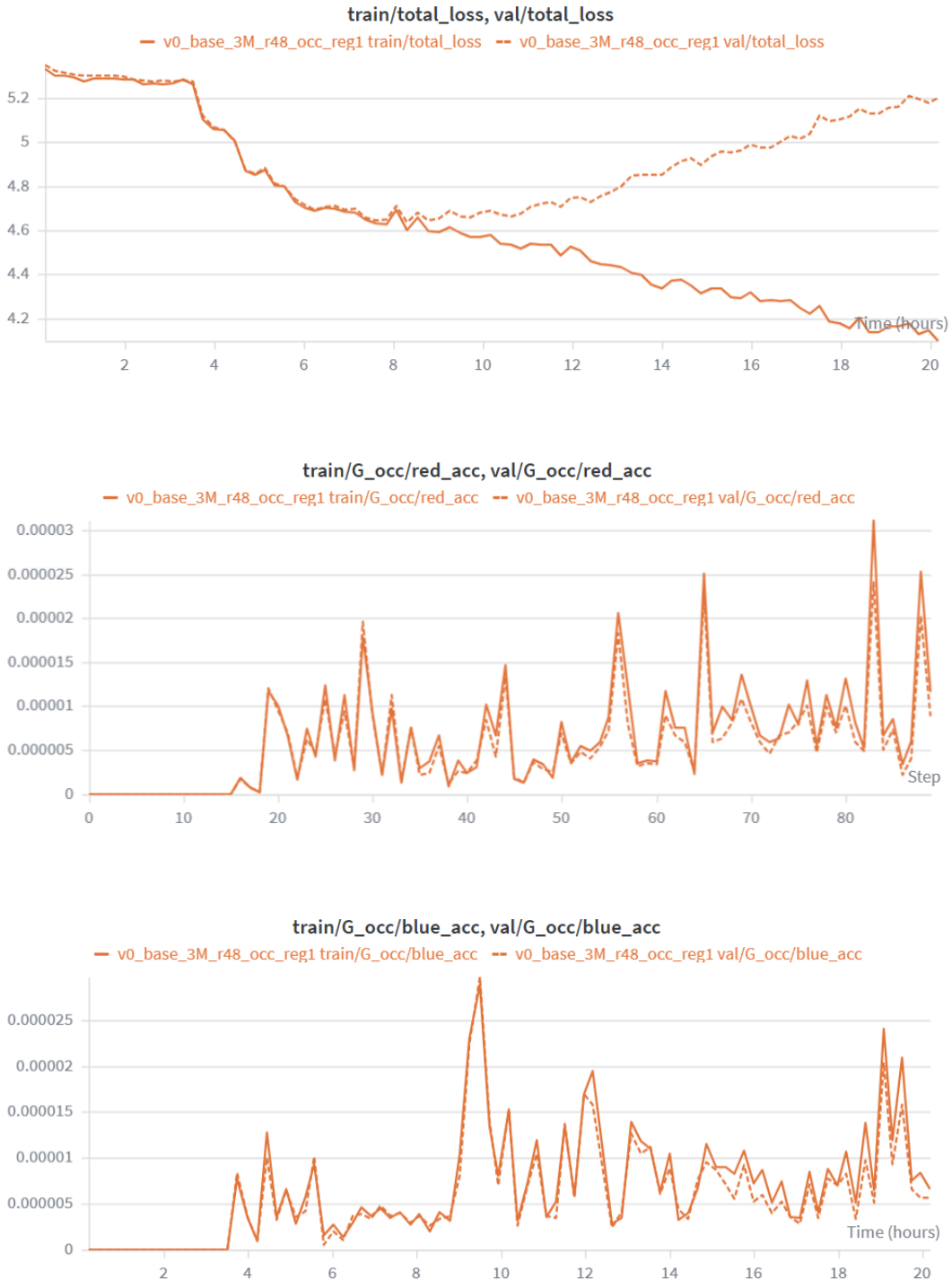

Figure 21: *Top*: Train/Val total loss; *Middle*: Train/Val Red occupancy accuracy for ground grid; *Bottom*: Train/Val Blue occupancy accuracy for ground grid. We see total loss diverge and low occupancy accuracy < 0.00003%.

To mitigate training instability, we replace vanilla CA with Deformable CA (Zhu, 2021). In short, Deformable CA constrains queries to interact with only a small subset of key and value pairs instead of all, like CA does. The rule for this subset constraint is problem specific; in our case, we can use the camera projection matrix to restrict this interaction. Specifically, since we know camera intrinsics and extrinsics, we can project any point in output space back to image space as a reference point. Then, we can restrict query q_i to only interact with $\{k_i, \ldots k_{i+n}\}$, $\{v_i, \ldots, v_{i+n}\}$ around some reference point in the image. We saw much faster convergence and stable training by using Deformable CA (Figure 22).

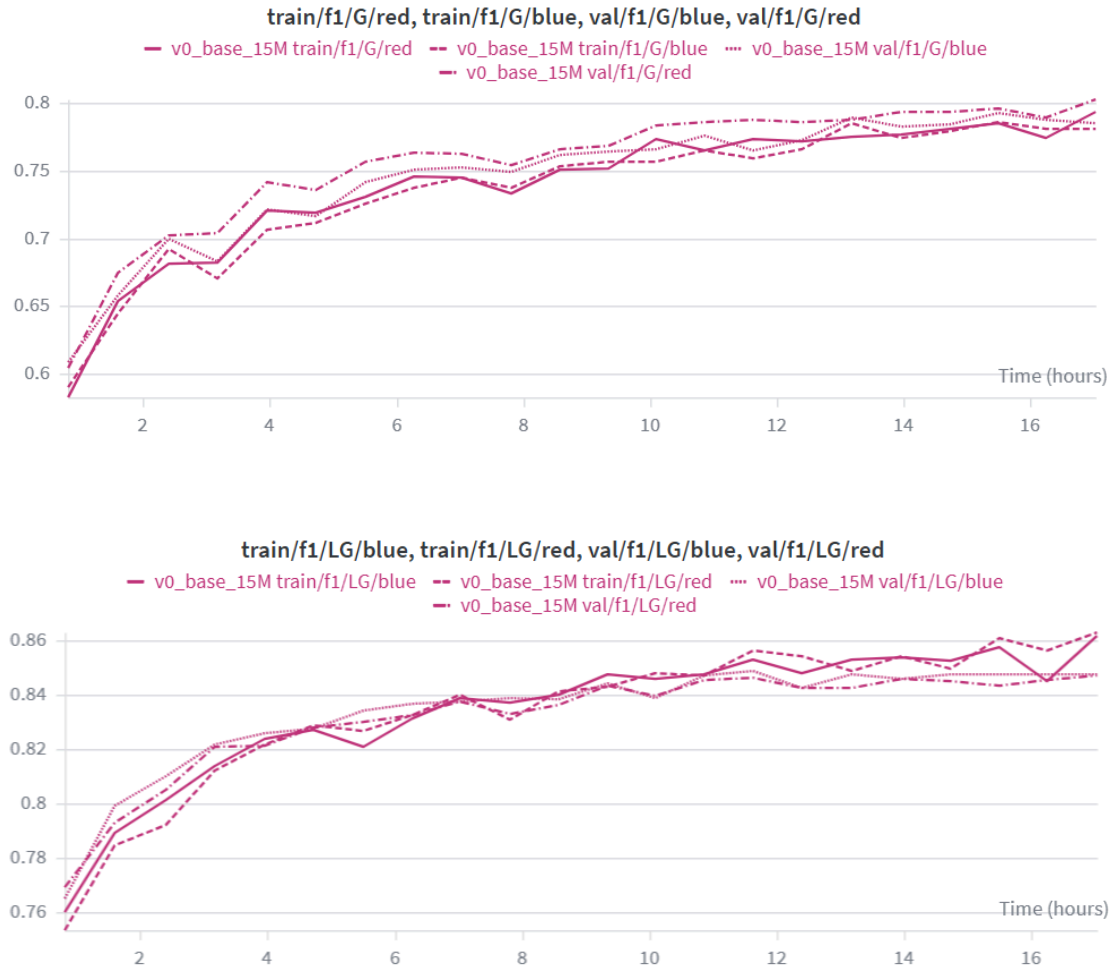

Figure 22: Left: Train/Val F1 score for red/blue in ground grid. Right: Train/Val F1 score for red/blue in Long Goal grid.

To reduce the effect of the labelling density data imbalance we experimented with several compensatory strategies in the model and training architecture:

1. **Weighted cross-entropy**: Set higher weights to occupied cells.
2. **Over/Under-Sampling**: Drop null labels in each batch to reduce imbalance ratio.

3. **Different Losses**: Focal loss, dice loss, IOU loss, and other auxiliary loss such as block count.

After replacing CA with Deformable CA we only kept over/under-sampling as a balancing strategy, since the others did not significantly improve training. However, theoretically, with careful tuning they still could have provided benefits.

Evaluation

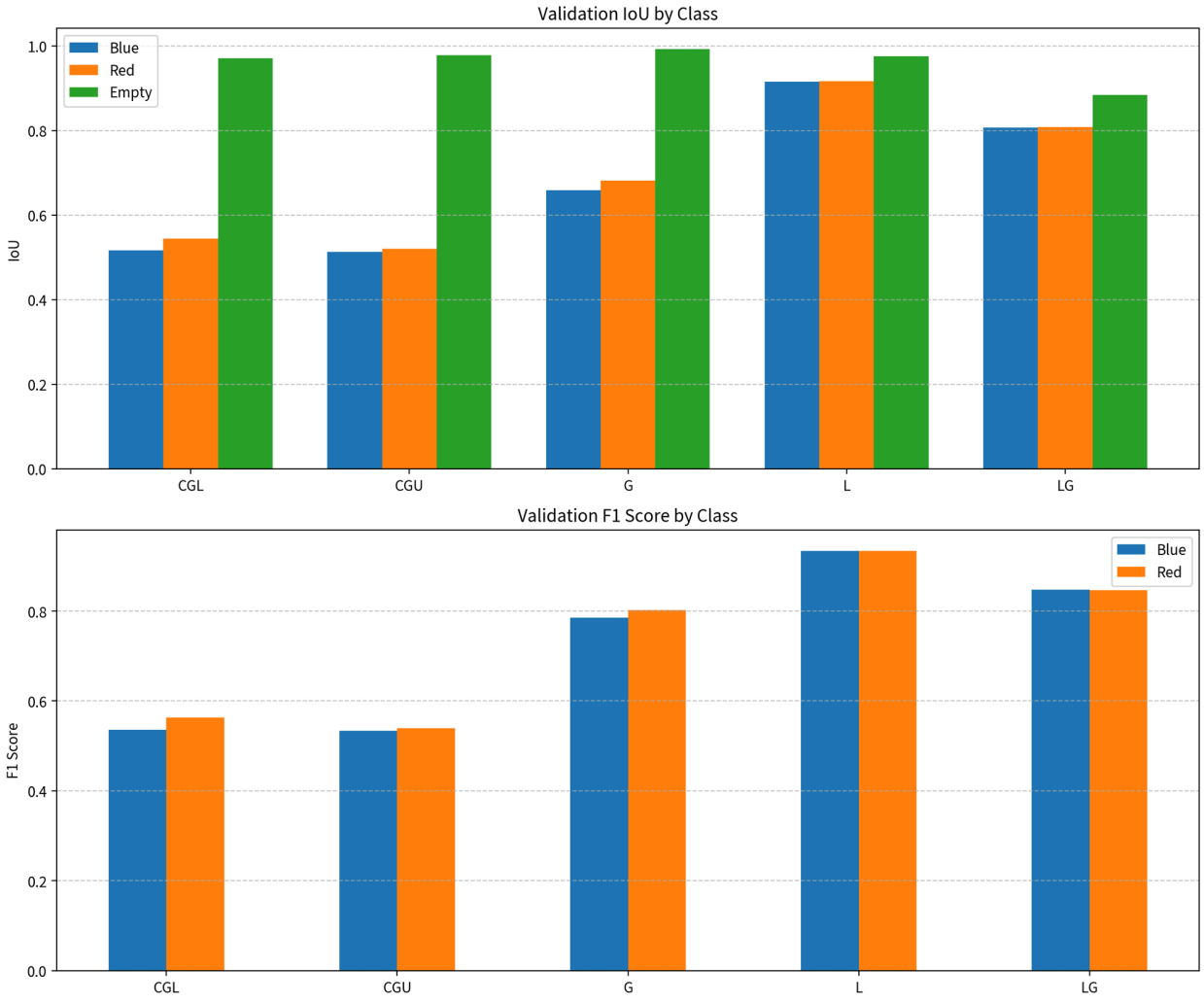

Figure 23: IOU and F1 Score on Validation set (~20k samples).

The model was evaluated using the F1 Score and IOU, calculated on each grid type. We found that the center goal grids performed worse than the other, possibly that this happens due to occlusion and glare (Figure 23).

3D Coordinate Prediction

While our previous model performed well, by nature it is imprecise due to the discrete formulation. With a 48x48 grid, the maximum distance error of prediction is 3" for a correct prediction, and the number is of course larger for mispredictions. This is a slight issue for downstream layers in the robot software, since picking up blocks requires precise line up. In addition, this is a bigger issue for Strategy AI which is sensitive to noise. We could naively increase the grid resolution consequently lowering maximum distance error at the cost of quadratic increase in computation (due to attention in the Transformer). Instead, we reformulated the object detection task from discrete occupancy prediction to 3D coordinate prediction, directly regressing block center and bounding box.

We kept the majority of the previous architecture but changed the detection head from classification to regression, and added a Transformer Decoder for processing positional query and object query. The detection head now outputs $(x, y, z, d_x, d_y, d_z, c)$ corresponding to 3D location, bounding box size, and colour. Although the bounding box size is not theoretically needed, we found it helps learning speed. We also doubled the parameter count of our model, due to additional Transformer modules (Figure 24).

Since the task is now formulated as bounding box prediction, we can no longer use the BEV plane or grids as input to detection heads. Instead, similar to the idea of positional queries, we use object queries, which are learnable. In the Transformer Encoder, image features are used

as conditioning, and in the Transformer Decoder, positional queries are used as conditioning. We use in total 88 object queries, since there are at most 88 blocks.

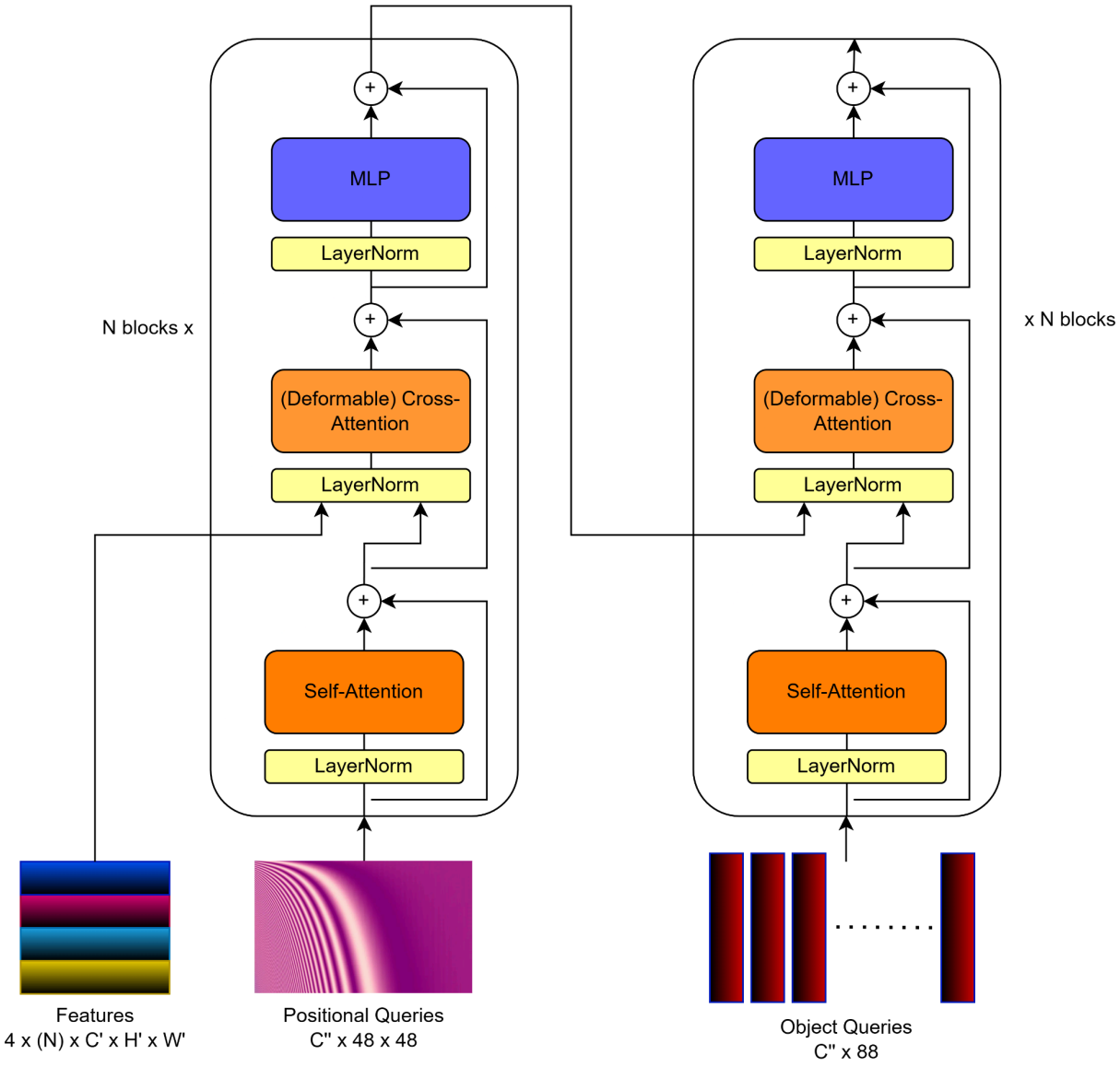

Figure 24: Complete Transformer modules for 3D coordinate prediction

Iterative Refinement Detection Head

We follow the DETR3D formulation of the detection head by implementing iterative coordinate refinement instead of direct one-shot prediction. More specifically, we predict block

location at every decoder layer, then use these coordinates as reference points on positional queries for Deformable CA constraints. This improves accuracy by ~33% (from ~3cm to ~2cm L2 error).

Hungarian Loss

Instead of classification, our task is now a set-to-set prediction task. We followed common practice and used Hungarian Cost Matching as our loss function. Our model produces a fixed set of $N = 88$ predictions. These are matched to the set of M objects ($M << N$) using the Hungarian algorithm (bipartite matching). For each prediction i and ground-truth object j, the matching cost is:

$$\mathcal{L}_{\text{match}}(i, j) = \lambda_{\text{cls}}\mathcal{L}_{\text{cls}}(\hat{c}_i, c_j) + \lambda_{\text{box}}\mathcal{L}_{\text{box}}(\hat{b}_i, b_j)$$

Where:

- $\hat{c}_i$: predicted class probability (red or blue)
- c_j: ground truth class
- $\hat{b}_i$: predicted 3D box (center, size)
- b_j: ground truth 3D box

And:

- $\mathcal{L}_{cls}$: cross entropy
- $\mathcal{L}_{box}$: L1 loss + IOU

The Hungarian assignment (solution to cost matching) is:

$$\hat{\sigma} = \arg\min_{\sigma \in \mathfrak{S}_N} \sum_{i=1}^{N} \mathcal{L}_{\text{match}}(i, \sigma(i))$$

Where $\mathfrak{S}_N$ is the set of all possible permutations. We solve this in code using torch.cdist and scipy. After optimal matching is found, the final loss is:

$$\mathcal{L}_{\text{Hungarian}} = \sum_{i=1}^{N} \left[\mathbf{1}_{\{\hat{\sigma}(i) \neq \emptyset\}} \left(\lambda_{\text{cls}} \mathcal{L}_{\text{cls}}(\hat{c}_i, c_{\hat{\sigma}(i)}) + \lambda_{\text{box}} \mathcal{L}_{\text{box}}(\hat{b}_i, b_{\hat{\sigma}(i)}) \right) + \mathbf{1}_{\{\hat{\sigma}(i) = \emptyset\}} \lambda_{\text{noobj}} \mathcal{L}_{\text{cls}}(\hat{c}_i, \emptyset) \right]$$

We set $\lambda_{cls} = 1.0, \lambda_{box} = 0.3, \lambda_{noobj} = 0.1$.

Evaluation

Since the model outputs N predictions for $M << N$ ground truth objects, we use greedy matching to evaluate. A prediction is considered a match if:

$$|center_{pred} - center_{truth}| <= threshold$$

We evaluated at different thresholds of 1cm, 2cm, 4cm, 8cm, infinity. In addition, we also evaluated our model's prediction on different ranges (distance from robot). We use ranges 0-80cm, 80-160cm, 160-320cm, 320-640cm. Refer to Figure 25, 26.

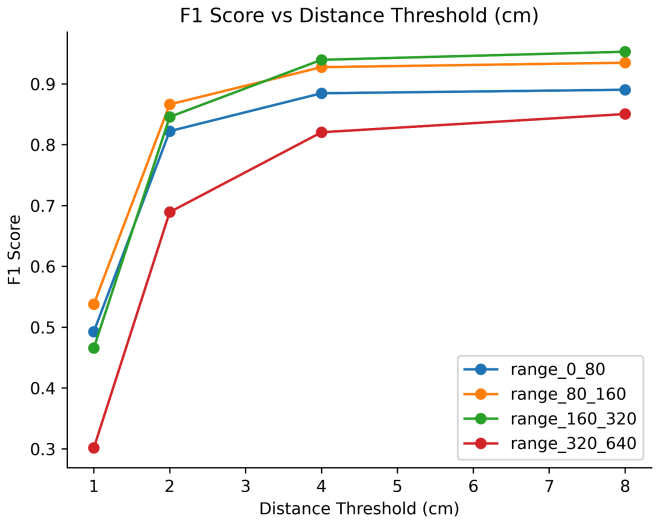

Figure 25: F1 Score vs Distance Threshold. At closer range, the model performs worse, due to blind spots from the cameras. Many objects in this range are either occluded or partially visible.

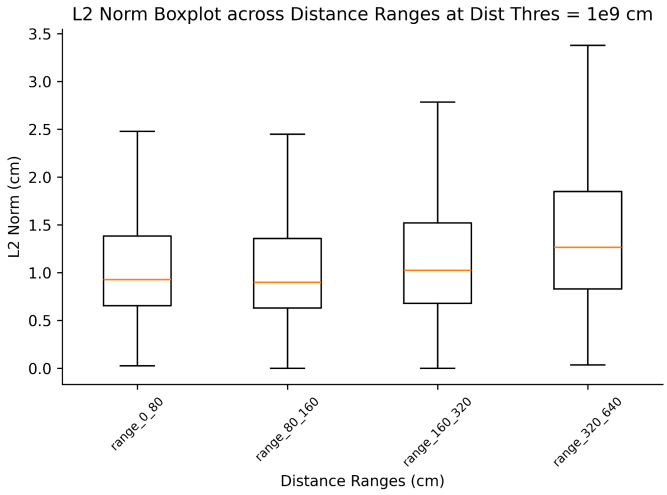

Figure 26: L2 norm with infinite threshold. Even with an infinite threshold, the median L2 norm across range stabilizes under 1.5cm.

Computer Vision - YOLO Block Position Mapping

Our mid-development change in cameras introduced a mismatch between the trained unified game state model and the new cameras. Retraining the model at scale was not feasible with the given time constraints. As a result, we transitioned to a geometric mapping pipeline using YOLO detections from the four corner cameras. After the YOLO model detects the game objects in the camera frame, we need to project the bounding boxes into the physical 3D space to find their position relative to the robot. This spatial mapping was done with a combination of camera intrinsic calibration and ground-plane projection.

To simplify the scope of this position mapping due to time constraints, we assumed that all blocks are on the ground. As a tradeoff of this assumption, we cannot perceive data about blocks in goals or loaders. Blocks in goals or loaders will appear to be outside the field and be ignored. From each detected bounding box, we choose one pixel point (u,v), the bottom-center of that bounding box, that we assume to lie on the ground plane:

$$u = \frac{x_1 + x_2}{2}, \quad v = y_2$$

Each camera was calibrated using a checkerboard to obtain the intrinsic camera matrix and lens distortion coefficients. Due to the wide-angle lenses, we found radial distortion in the image, particularly near the edges. To address this, detected pixel coordinates are undistorted using the calibration parameters. This converts the image pixels into normalized camera rays, producing more accurate directional vectors from the camera lens.

$$\mathbf{r} = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

Ground Plane Projection

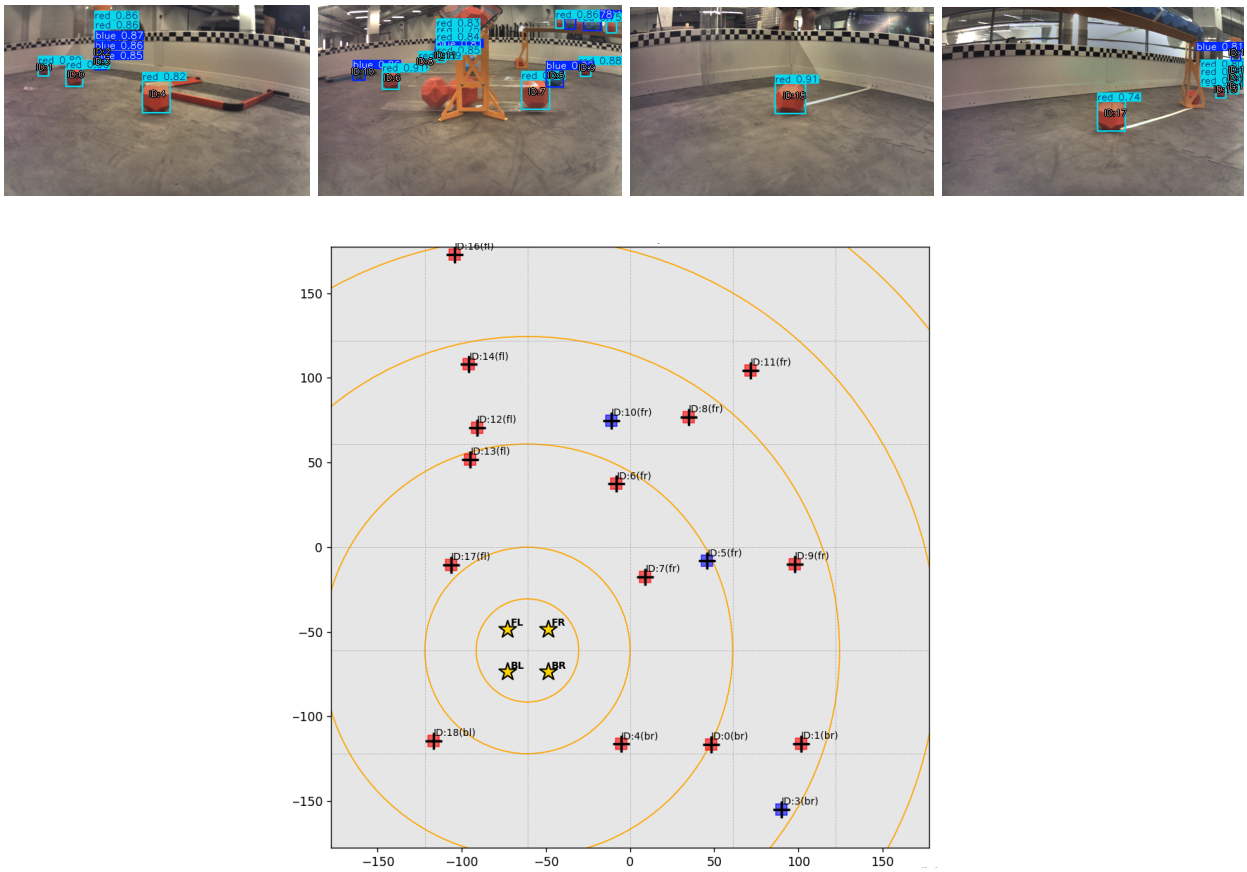

Figure 27: Bounding boxes from YOLO mapped onto field positions

By assuming that all blocks rest flat on the field surface, we can project the undistorted 2D point onto the ground plane. Each camera is mounted at a known height h=8in and pitch θ=-15°. With this we can calculate the distance from the camera's optical center to the block along the camera's optical axis Z_c:

$$Z_c = \frac{h}{y' \cos\theta + \sin\theta}$$

Using Zc we calculate the block's physical position relative to the camera (xlocal, ylocal):

$$x_{\text{local}} = Z_c \cdot x'$$

$$y_{\text{local}} = Z_c \cdot (\cos\theta - y' \sin\theta)$$

Each camera has a known yaw rotation $\boldsymbol{\psi}$ and position offset (ox,oy) from the robot's center. We transform the block positions from camera frame (xlocal,ylocal) into the robot frame (xr,yr):

$$x_r = x_{\text{local}} \cos\psi + y_{\text{local}} \sin\psi$$

$$y_r = -x_{\text{local}} \sin\psi + y_{\text{local}} \cos\psi$$

Then with the robot's position on the field, we can translate these points to field coordinates (xfield,yfield):

$$x_{\text{field}} = x_r + x_{\text{robot}}$$

$$y_{\text{field}} = y_r + y_{\text{robot}}$$

Learned Residual Neural Network Correction

While the geometric ground-plane projection was a solid baseline, through testing we observed that there were significant large errors of position of blocks that were at further distances and near the edges of the camera images. These errors are likely due to non-linear lens warping that our undistortion calibration could not fully correct.

To address this without significantly increasing inference time, we developed a small Multi-Layer Perceptron (MLP). Instead of predicting the field coordinates from scratch like the unified state model, this network acts as a corrector to adjust the geometric projection. It takes

the mathematically estimated coordinates and applies a learned spatial shift to correct the optical distortion. The corrected local camera frame position (xc,yc) is defined as:

$$(x_c, y_c) = (x_{\text{local}}), y_{\text{local}}) + f_\theta(u, v, c_x, c_y, x_{\text{local}}, y_{\text{local}})$$

where (cx,cy) is the principal point of the camera intrinsic matrix. The camera local coordinate is refined by a learned function $f\theta$, parameterized by a neural network, which predicts a residual correction conditioned on both image-space and geometric-space inputs.

To generate the training data for this model, we built a custom interactive graphical user interface (GUI) that overlays the coordinate projections onto a top-down Bird's-Eye View (BEV) grid.

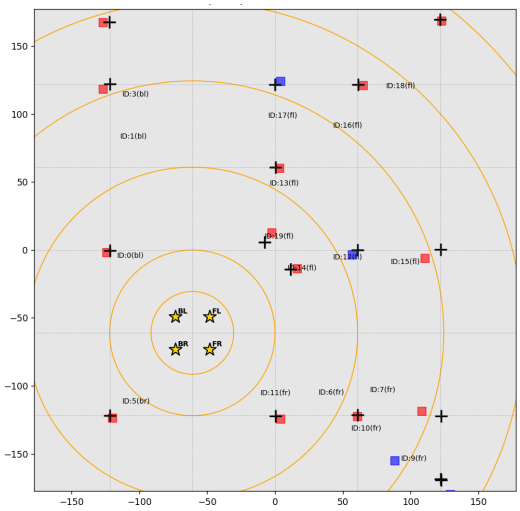

Figure 28: Block positions shown in the BEV GUI where the black "+" is ground truth

This GUI allowed us to run inference with the YOLO model, visually see the results, then drag the inaccurate projections to their true position to serve as the ground truth as seen in Figure 28. The user given ground truth field position (xt_field,yt_field) is converted back into the corresponding local camera frame using the inverse-rigid body transform:

$$(x_{t_{\text{local}}}, y_{t_{\text{local}}}) = R(-\psi)\big((x_{t_{\text{field}}}, y_{t_{\text{field}}}) - (o_x, o_y)\big)$$

where $R(-\psi)$ is the planar rotation matrix defined by the camera yaw ψ, and (ox,oy) are the positional offsets of the camera relative to the robot's center. From this we can find the residual error:

$$(\Delta x, \Delta y) = (x_{t_{\text{local}}} - x_{\text{local}},\ y_{t_{\text{local}}} - y_{\text{local}})$$

The MLP itself is implemented as a fully connected network with two hidden layers. The input is the feature vector $[u, v, c_x, c_y, x_{\text{local}}, y_{\text{local}}]$, and it outputs a 2D correction $(\Delta x, \Delta y)$. The network is trained to minimize the mean squared error between its predicted correction and the true residual error from the ground truth. To ensure stability during continuous training, the dataset includes both the corrected ground truth points from the GUI, and the unmodified points. For the unmodified points, the network preserves its previous predictions. This helps stabilize the training, and ensures that training with new data does not cause the model to forget its previous correct predictions. Because this model is so compact and lightweight, we are able to train it individually for each camera. Figure 29 shows how each detection is tied to the camera that detected it.

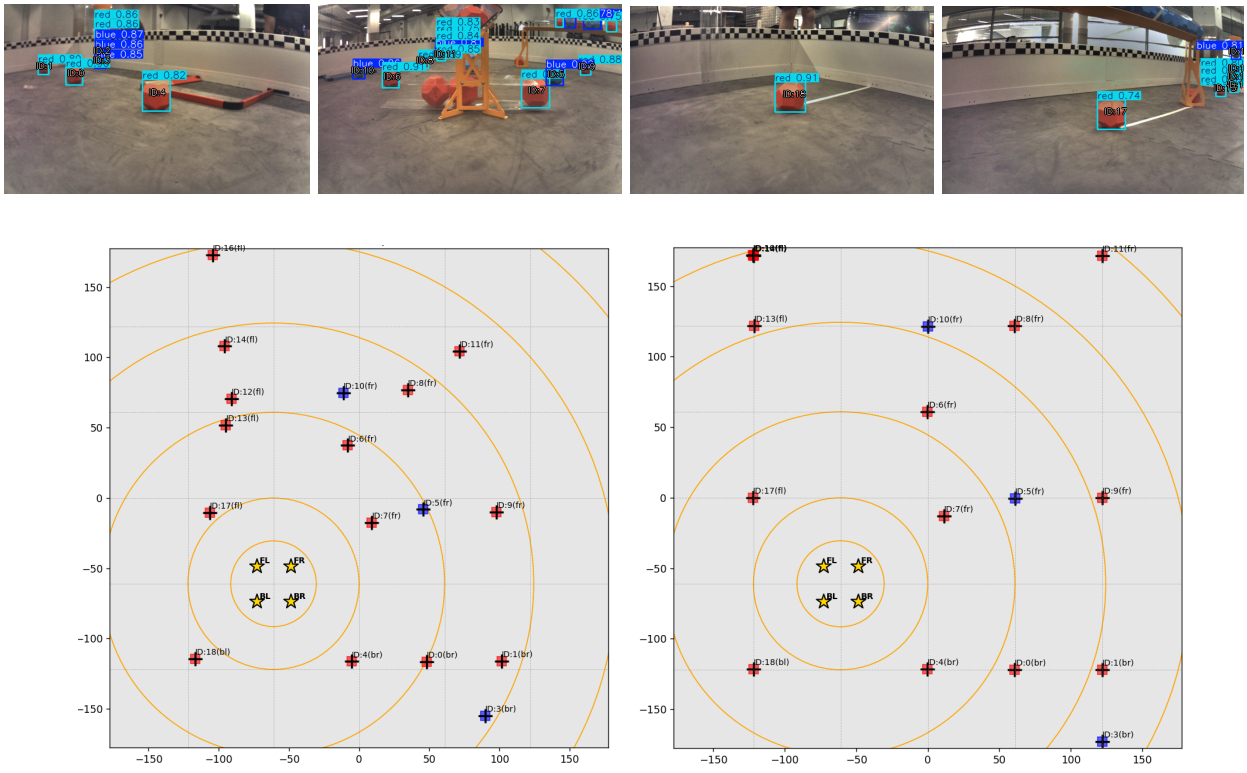

Figure 29: Block positions shown in the BEV GUI before (left) and after (right) the residual neural network correction

Strategy AI

Strategy AI acts as the “brain” behind decision-making, it must ingest as much information from the field as possible to output specific actions for the robot. To build and train such an agent, the first step is to implement it within a simulation to train the model’s in. This required us to build a custom simulation environment, design the model, and write a training pipeline. Over the course of development, we went through three major iterations:

1) We began by building a simulation in 3D space and trained a CNN using self-play.

2) We simplified the simulation to operate using various 1D arrays and rebuilt the field accordingly. We also evolved the model to use ResNets rather than CNNs and introduced "league players".
3) We eventually discovered that because the previous simulation used discrete space, the model learned "wrong" ideas about the world. Consequently, we developed a new simulation in continuous space to better mirror the real world, transitioned the input matrix from 3 dimensions to 1, and integrated evaluation metrics.

Iteration 1 - Discrete Simulation and Synchronous Training

At the start of development, the plan was for the agent to utilize spatial intelligence about the field by representing the environment as a 3D data structure. This required the simulation to either exist in 3D space or be translated into a 3D representation to be processed by the neural network. We chose to build the simulation using a 3D matrix where the environment logic was handled internally rather than separating individual components into their own data structures. While this seemed like a logical starting point, we eventually realized that this architecture caused the simulation to run too slowly for effective training, which did not become apparent until later in the development cycle.

Simulation

The simulation operated in a discrete space using a 48x48x6 grid. This specific dimensionality was chosen for several technical reasons. Primarily, we aimed for a resolution where each game block occupied at least one grid cell. This was important for the agent to accurately track block counts and positions within the goals, as precise placement determines the

final score. For instance, the robot needed to identify which blocks were in the control sections of the long goals to calculate how many additional blocks were required to dominate that zone. Given a block diameter of approximately 3.25 inches and a 12x12 foot field, the width and height could accommodate roughly 45 blocks; we utilized 48 cells for efficient GPU computation. The height of 6 cells was chosen to represent the maximum capacity of the blocks in the loaders..

We encode different entities as floating points in the grid. See Table 1 and Figure 30:

Value	Object
0	Empty Cells
0.66	Goals
0.33/-0.33	Alliance and opponent blocks on the field
0.25/-0.25	Alliance and opponent parking zones
1.00/-1.00	Alliance and opponent robots

Table 1: Object Value Mapping

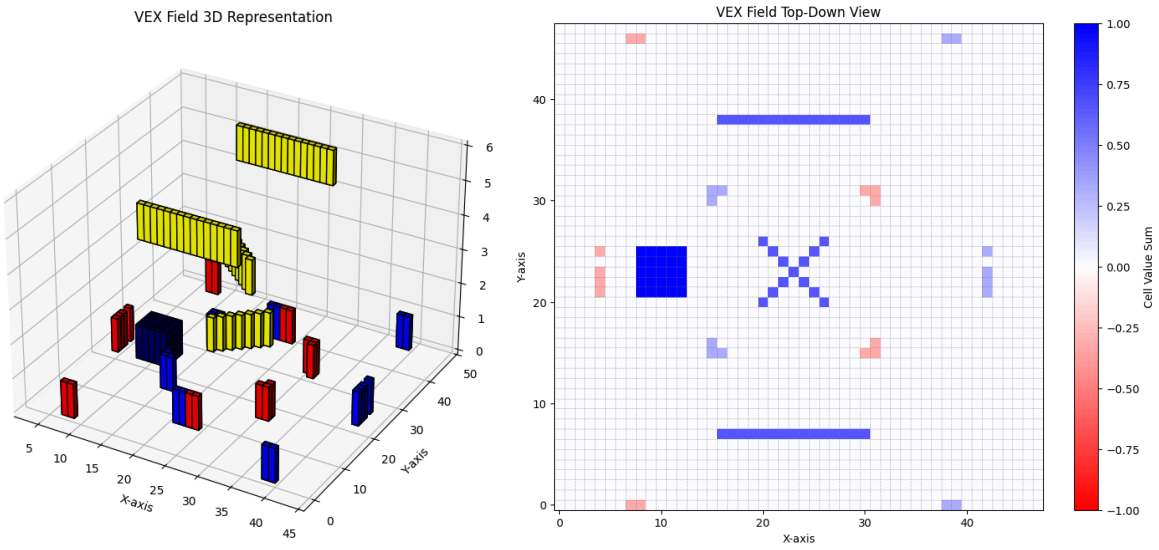

Figure 30: The left figure displays a 3D spatial representation of the environment, identifying the alliance robot and corresponding blocks in blue, opponent robot and blocks in red, and goals in yellow. The right figure provides a top-down view of the field, using a color-coded gradient scale bar to indicate object types.

Training

To preserve the spatial intelligence of the environment, we initially opted for a Convolutional Neural Network (CNN) architecture, outputting a policy and value heads. This model consisted of approximately 260,000 parameters and was designed to extract local features from the 3D grid, such as the proximity of blocks to the robot or the alignment of the robot with the goals. We chose this size to maintain a high inference speed during simulation, but we eventually realized that the model lacked the capacity to represent complex, long-term strategic decisions.

Since we lacked a pre-programmed opponent or a human player to train against, we implemented a self-play architecture. This allowed the model to improve by competing against

its own current version, effectively doubling the amount of relevant data generated per session and ensuring the agent always faced an opponent of equal skill. To make this work without training separate models for different sides of the field, we utilized a coordinate transformation system. We flipped the field representation for the opponent's perspective so the model always perceived itself as starting from the same relative position. Furthermore, we inverted the numerical values of the entities so that negative values representing the opponent were converted to positive values, allowing the model to use the same logic for decision-making regardless of its assigned color or starting coordinate as shown on Figure 31.

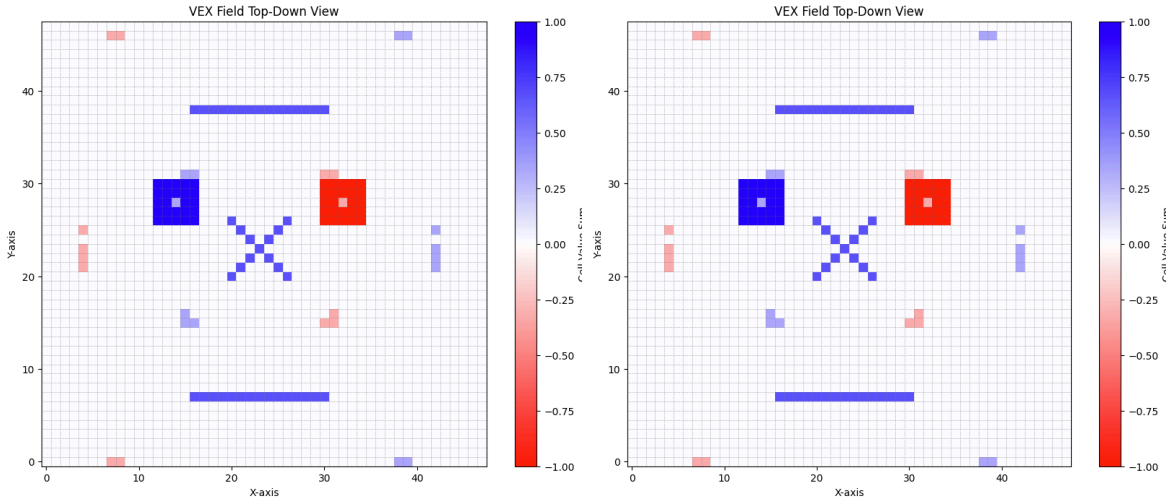

Figure 31: The left figure displays the field from the alliance perspective, where blue indicates positive values for the robot and its associated blocks , and red indicates negative values for the opponent. The right panel demonstrates the result of the map flipping process; when the perspective shifts to the opponent, the environment is inverted so that the red robot is now treated as the alliance agent. This transformation makes the field representation identical to the original alliance view, allowing the model to process both roles.

We used Proximal Policy Optimization to train our agent. Standard policy gradients (REINFORCE) are notoriously difficult to tune because a single optimizer step can result in a massive policy shift that collapses the model's performance. PPO addresses this instability through two primary mechanisms. It maintains a distinct ratio between the new and old policies and applies a clipping function to the probability updates. This keeps the updates within a specific range and prevents the model from making overly aggressive changes to its strategy. Here is the general formula:

$$r_t(\theta) = \frac{\boldsymbol{\pi}_{\theta}(a_t|s_t)}{\boldsymbol{\pi}_{\theta old}(a_t|s_t)}$$

$$L(\theta) = \hat{\mathrm{E}}_t[min(r_t(\theta)A_t,\ clip(r_t(\theta),\ 1-\epsilon,\ 1+\epsilon)A_t]$$

To guide the learning process, we implemented a combination of intermediate and final rewards. Relying solely on a sparse final reward often makes it difficult for the reinforcement learning algorithm to determine which specific actions at earlier timesteps contributed to a successful outcome. By introducing intermediate rewards, we provided the model with more immediate feedback. Specifically, the agent received positive reinforcement for picking up blocks and scoring them into goals, with a higher reward value assigned if the blocks were placed in the control zones. We use discounted rewards for credit assignments. This approach helped the agent prioritize immediate gains while still accounting for future potential. These values were then used for advantage estimation, as shown below:

$$G_t = \sum_{k=0}^{T-t+1} \gamma^k r_t$$

$$A_t = G_t - V(s_t)$$

Iteration 2 - Improved Discrete Simulation and Synchronous Training

After training the first iteration for approximately 92,000 steps over the course of a full day, it became clear that the development cycle was too slow. We identified that the simulation logic was the primary bottleneck, as executing a single action within the environment required more time than the actual neural network update. To resolve this, we restructured the simulation by decomposing it into simpler components rather than performing all operations within the 3D space. The optimized simulation allowed us to dramatically increase the simulation speed and provided the foundation for more rapid experimentation. Along with these performance gains, we refined our model architecture and improved the training pipeline to further accelerate the learning process.

Simulation

To maximize simulation speed, we refactored the environment by decomposing each element into a dedicated class. We developed independent classes for the robots, the long and center goals, the loaders, and the blocks located on the ground. This modular approach allowed each class to maintain its own optimized data structure, which we then aggregated into a single 3D tensor with dimensions of 48x48x8. In addition to the spatial data, we introduced two extra layers to the tensor to provide the model with critical state information. One layer represented the current robot's inventory, while the other represented the remaining time. We normalized both of these values to fall between 0 and 1

Training

We implemented several technical upgrades to the training process to improve speed and stability.

ResNet architecture

We transitioned from a standard CNN to a ResNet architecture to leverage residual connections. These connections allow the network to pass information directly across layers, which prevents the loss of critical spatial data and helps mitigate the vanishing gradient problem during deep training. The new model also was scaled up to around 1M parameters.

General Advantage Estimation (GAE)

Our previous implementation relied on a basic discounted reward calculation, but we found this approach produced high variance and noisy updates. Therefore, we integrated Generalized Advantage Estimation (GAE) using temporal difference (TD) learning. GAE serves to smooth out noisy reward signals by using a λ parameter to balance the reliance on the critic's value predictions against the actual rewards received. The temporal difference error at each step represents the immediate surprise or discrepancy in the model's prediction, calculated as:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t),\ \gamma = 0.99$$

The final advantage at any given timestep is an exponentially weighted moving average of these TD errors, which allows the model to better account for long-term future outcomes while controlling for variance:

$$A_t = \delta_t + (\gamma\lambda)A_{t+1},\ \lambda = 0.95$$

League Players

If an agent exclusively competes against its most recent version, it may gradually lose the ability to defeat earlier or different types of strategies (Balduzzi, 2019. To avoid this regression, we implemented a league-based system that maintains a history of model snapshots in a dedicated class. Specifically, we configured the training pipeline so that the agent faced the current version of itself 80% of the time to continue pushing the performance ceiling, while the remaining 20% of matches were played against older snapshots. We used a dynamic sampling system in which each past opponent $i = 1 \dots N$ is given a quality score q_i. Opponents agents are sampled according to a softmax distribution. Snapshot (past agent) i is chosen with probability p_i proportional to e^{q_i}. We add current agent to past agent pool every 10 iterations and initialise its quality score to maximum of all existing qualities. If a past opponent defeats the current agent, no update is applied. If the current agent defeats a past opponent, we update the past opponent quality using the formula below.

$$q_i \leftarrow q_i - \frac{\eta}{N_{p_i}}$$

Where η is the tunable learning rate.

Multithreaded Rollout Generation

To further increase training throughput, we implemented multithreaded rollout generation. This involved deploying a shared model across 64 CPU cores, with each core acting as an independent worker. These workers operated in parallel, each playing a full game from start to finish. During these episodes, the workers recorded the model outputs, environment states, and chosen actions, storing this data in a shared buffer. As the central model was updated, the workers synchronized with the latest version simultaneously. The setup allowed the trainer to

pull large volumes of high-quality experience from the buffer for optimization, significantly reducing the time required for each training cycle.

Evaluation

We implemented an asynchronous evaluation system to track model progress using the TrueSkill rating system. By utilizing 8 to 16 dedicated evaluation workers on separate CPU cores, we could continuously benchmark the current model against a pool of up to 15 previous versions. TrueSkill assesses performance by maintaining a probability distribution defined by a mean, representing the perceived skill level, and a sigma, representing the degree of uncertainty in that rating. We continuously evaluated these values to maintain a high quality of training data. Before initiating a rollout, the system would match the current model with a previous snapshot only if the calculated win probability fell between 0.15 and 0.85. This range ensured that the matches were neither too easy nor too difficult, focusing the model's learning on competitive scenarios that provided the most useful strategic feedback.

Iteration 3 - Continuous Simulation and Asynchronous Training

In the second iteration, we identified a significant discrepancy between the agent’s behavior and real-world physical constraints. The model did not fundamentally understand the cost of movement. For example, the simulation allowed the robot to collect up to four blocks simultaneously simply by moving near them, failing to account for the time required to rotate and align with each individual block. Furthermore, our representation of scoring logic was overly simplistic. In the real world, pushing a block into a goal imparts force and velocity, often causing the block to settle in unpredictable positions. Our simulation initially stopped the block at the

corner of the goal grid; even after we introduced a randomization factor across a 3-4 cell radius, it still failed to capture realistic environmental dynamics. Therefore, we implemented a dedicated physics engine in the background of the simulation for the third iteration. This added a layer of realism to block trajectories and robot maneuvering, ensuring that actions in the simulation more closely mirrored real-world.

Because the introduction of a physics engine increased the computational load and slowed our training, we shifted to a fully asynchronous training architecture as a trade-off. By decoupling the rollout generation from the model optimization, we were able to maintain high training efficiency despite the increased complexity of the simulation environment.

Simulation

To achieve a higher degree of physical accuracy, we integrated PyMunk as our primary physics engine. We selected PyMunk because it is a lightweight, 2D rigid-body library that allows for efficient collision detection and realistic movement dynamics. The field is represented in a continuous 2D space with dimensions of 356.6922x356.6922 units. Within this environment, the robot now operates with a defined heading, which forces the model to learn the importance of orientation. The agent must successfully align itself with objects to pick up blocks or score in the goals, mirroring the mechanical requirements of a physical robot. To further simulate real-world constraints, we locked the simulation to a 200-millisecond step interval. This delay represents the expected inference time of the model on hardware, teaching the strategy AI to make decisions that account for the latency it will encounter in the real world.

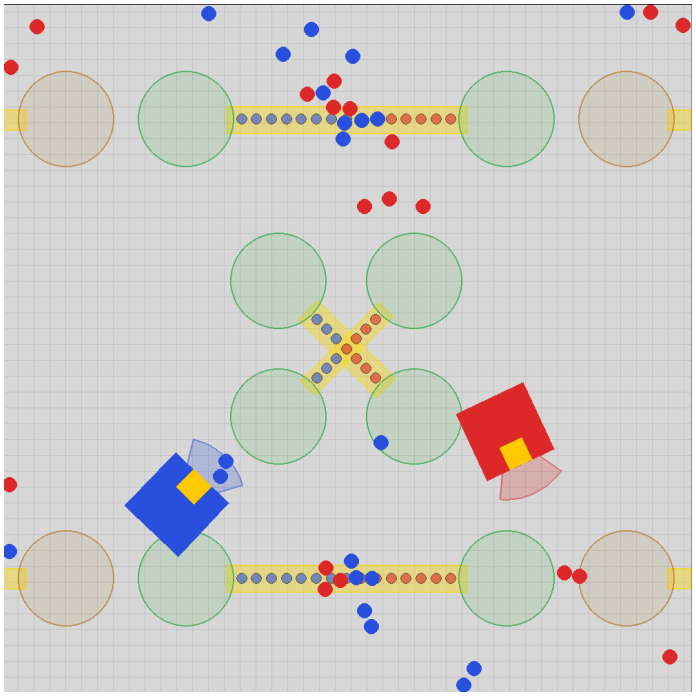

Figure 32: This figure illustrates the continuous space simulation. The circular regions define the areas where the robot is capable of scoring in the goals, loading blocks from the loaders, or picking up blocks from the ground. These boundaries represent the physical constraints the agent must navigate, requiring it to maintain proper alignment to simulate the real world.

Field Representation and Model

Upon transitioning to the 3D field representation with ResNet, we observed that a significant majority of the spatial tensor remained sparse, containing mostly zeros. This resulted in inefficient memory usage and wasted computational power. Inspired by the architectural efficiencies presented in the OpenAI Five paper, we moved away from the spatial grid in favor of a condensed 1D observation vector. This allowed the agent to process a higher density of information while simultaneously reducing the computational overhead.

For each observation, we integrated the following features into a unified 1D representation as shown in Table 2:

Category	Dimension	Feature Index	Description
Match Context	2	0 - 1	Time and score difference.
Field Control	12	2 - 13	One-hot vectors (3 each) for control of Bottom Long, Upper Long, Center Lower, and Center Upper goals.
Alliance	4	14 - 17	x, y and cos(θ), sin(θ) of the current robot.
Opponent	4	18 - 21	x, y and cos(θ), sin(θ) of the opponent robot.
Inventory	1	22	Alliance robot inventory size.
Score Difference	2	23 - 24	Own score and opponent score.
Static Objects	60	25 - 84	Data for 12 elements (4 Long Goals, 4 Center Goals, 4 Loaders). Each has 5 features: [dx, dy, dist, sin(err), cos(err)].
Block State	88×29	85+	Spatial (8): Abs. x, y, z; Relative dx, dy, dist; Heading sin, cos. Status (21): One-hot encoding of block location (Ground, Goals, Loaders, or Inventory).

Table 2: Observation space, all normalized between 0 and 1

Since the input was no longer a spatial grid, we replaced the ResNet with a simpler and more efficient Multi Layer Perceptron. The model output was structured to define primary actions such as blocking, scoring, picking up from a group, loading, or moving. While the physics simulation operated in a continuous space, we implemented a discrete output head for movement to stabilize the learning process (also known as parameterised action/policy). If the model elected to move, it outputted probabilities across a 20x20 grid representing a local 20x20 cm area around the robot. Similarly, the target heading was discretized into 72 distinct segments around the robot, with each segment representing 5 degrees rotation.

Training

As previously noted, the introduction of the physics engine decreased simulation speeds and extended the total training time. To address this, we transitioned from a synchronous training loop where workers would stop and wait for the optimizer to an asynchronous architecture. Asynchronous pipeline allowed the workers to generate rollouts and the trainer to update the model simultaneously as shown on Figure 33. However, we identified two primary issues with this implementation. Because the simulation workers produced data at a slower rate than the trainer’s consumption, the system encountered issues with sample reuse and staleness.

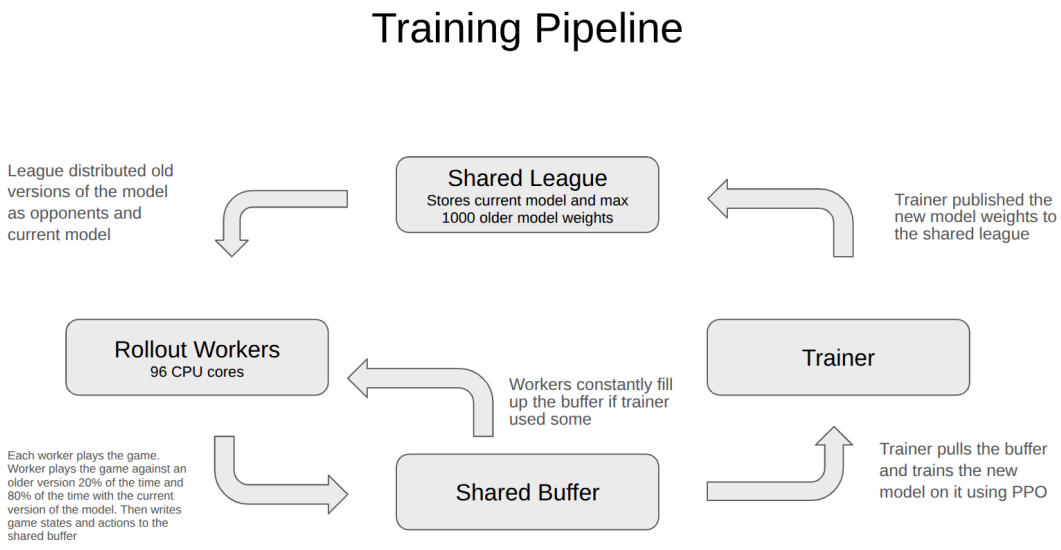

Figure 33: The diagram shows the asynchronous rollout generation using CPU and training using CPU.

Sample Reuse

Sample reuse refers to the number of times a single data point is used to update the model. High reuse rates can significantly slow down the learning process and lead to overfitting. In an ideal scenario, this value would be as low as possible, meaning every piece of data is used exactly once; while maintaining a value above 1 is generally considered ineffective as shown on Figure 34. We calculated sample reuse using the following ratio:

$$Sample\ Reuse\ = \frac{(sample\ per\ batch)(batches\ per\ second)}{buffer\ intake\ samples\ per\ second}$$

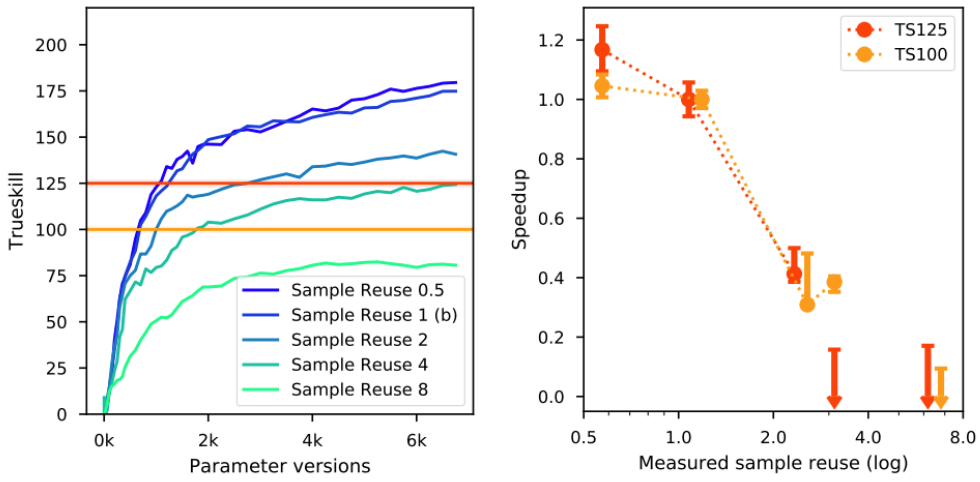

Figure 34: Sample Reuse effect on training speed and TrueSkill value (OpenAI, 2023)

By increasing the number of parallel rollout workers from 64 to 96, we ensured a more consistent flow of fresh data. This adjustment successfully brought our sample reuse down to approximately 1.05, providing the trainer with a near-constant stream of new data as shown on Figure 35.

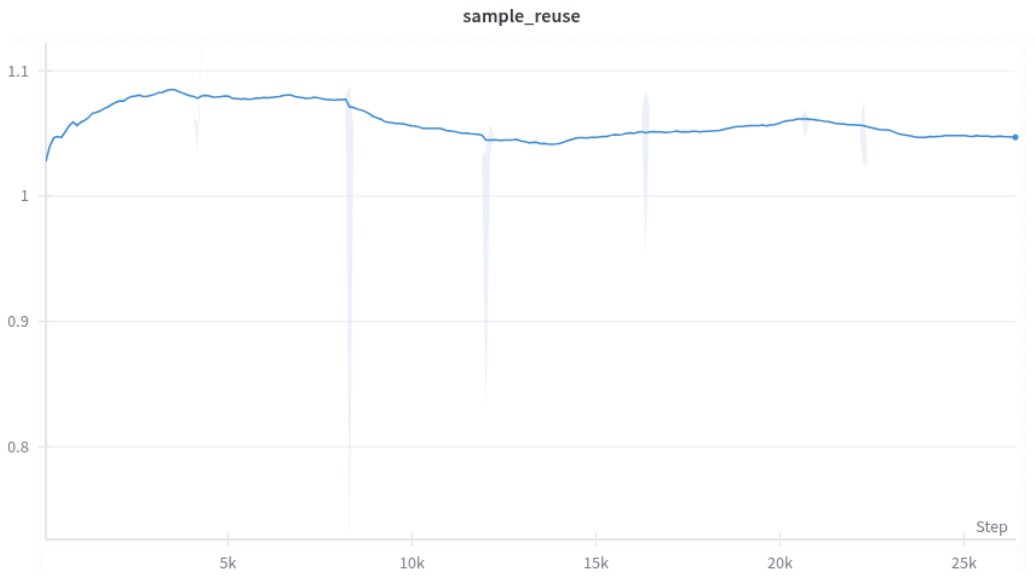

Figure 35: Sample reuse over 25000 timesteps

Staleness

Staleness measures how many versions behind a worker is, compared to the current model being optimized. If the staleness is too high, the trainer attempts to improve a policy using data that no longer represents the current behavior of the agent. As staleness increases, the quality of learning degrades as the gradients become less relevant to the current parameters as shown on Figure 36.

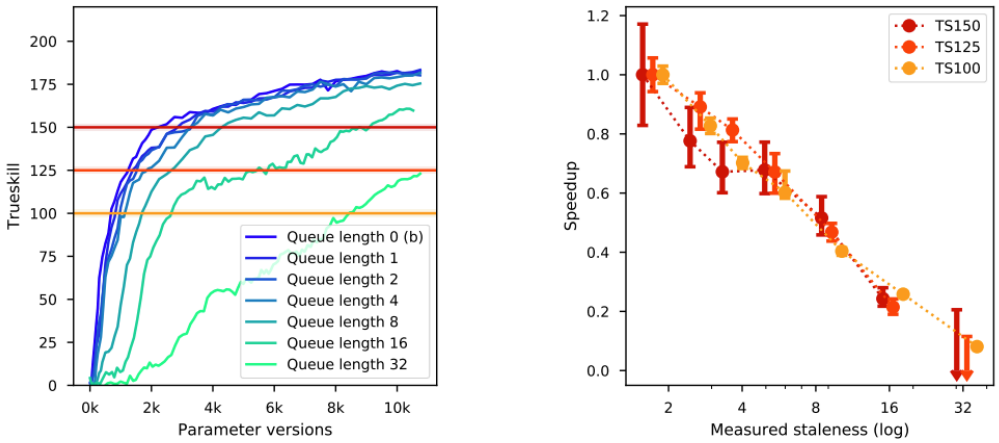

Figure 36: Staleness effect on training speed and TrueSkill value (OpenAI, 2023)

To avoid high staleness,we implemented a synchronization check that required workers to frequently verify if a newly optimized model was available. Although this constant polling introduced a slight delay in rollout generation, it was the most effective solution for maintaining policy alignment. Through this method, we achieved a staleness value of approximately 0.5 as shown on Figure 37.

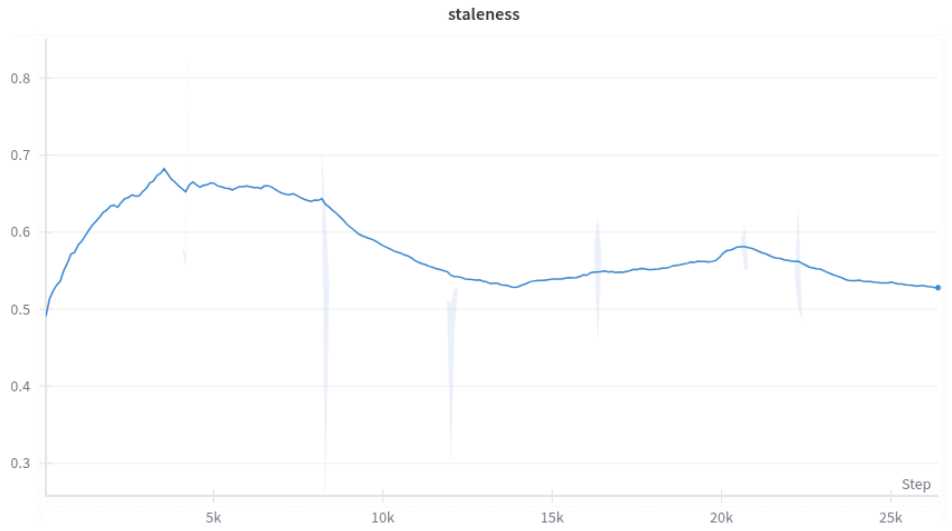

Figure 37: Staleness over 25000 timesteps

Results

The Strategy Model was trained for 8 days for 26M games using 96 workers and 6 evaluation workers over 102 CPU cores and RTX 6000 Blackwell for training. As shown on Figure 38, the model converged at around 160 TrueSkill value, although longer training could have led to higher value.

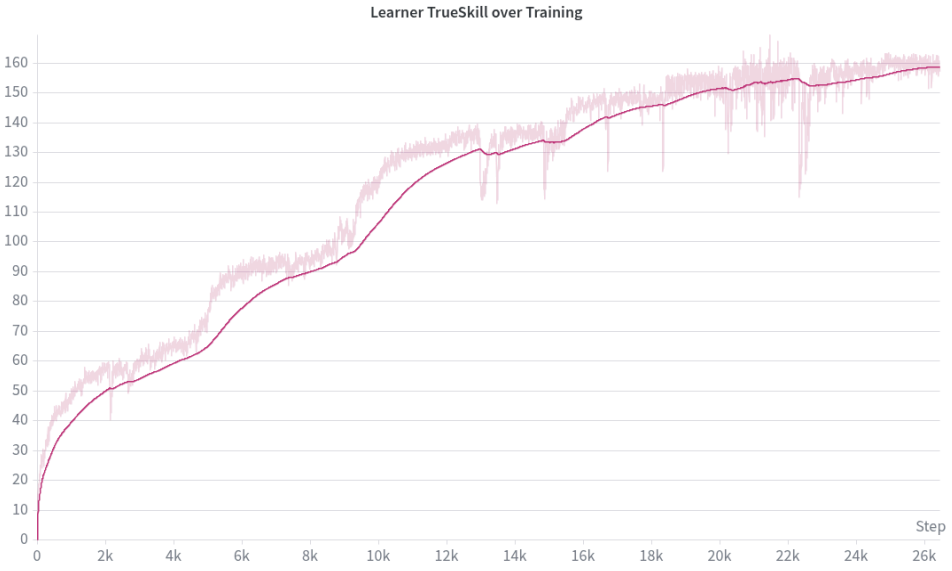

Figure 38: The plot demonstrates TrueSkill value over training for 26M games

Future Work

Robot Electronics

The performance, usability and particularly the reliability of the robot electronics as designed and built left a lot to be desired. The overall architecture, using the Jetson AGX platform as the central computer with LiDARs over ethernet, other sensors over I^2C, and the VEX V5 Brain over RS-485, is a solid design. However, there were critical execution errors in the comms board PCB, the connectors chosen were very fragile, and we never managed to connect all the cameras to the main Jetson simultaneously.

Across two revisions, every single comms board that was built eventually burnt out the RS-485 transceivers (which were the only "real" components on the board). These failures are probably due to lacking static protection, in the form of missing TVS diodes across the RS-485 A/B lines. This same design error plagued the VEX V5 Brain PCB itself for many years, and has a simple fix that we missed due to inexperience with PCB design. Additionally, the PCB is far bigger than it needs to be due to wasted board space for SPI and CAN connections that we thought might be used, but never were. It could be integrated into a much sleeker package that fits nicely onto the AGX Orin and does not stick out the bottom, which limits mounting options.

The power board performed well enough for our use-case, though we didn't reach an integrated enough final robot to really make great use of the functionality it was mostly designed around. It needs design or assembly tweaks to make the LED battery level indicator autodetection functionality work without I^2C configuration, and it may be better to integrate a full BMS into the board for use with raw cells instead of a strange floating fuel gauge with an external BMS. It also hasn't been validated for the high-current use-case of Jetson AGX Orin +

V5 Brain all running at max power, and it seems entirely possible that doing that would lead to a more tenuous thermal situation than we encountered.

The camera hardware was our biggest pain point in this project, though we got close to making it work. Likely the last piece that would be needed to achieve a reasonable setup with the camera solution that we chose is just a working adapter from the AGX Orin's 120-pin MIPI-CSI-2 connector to the typical CSI ribbon connector. These cameras do seem to work as-is, though their drivers are very fickle and would still likely need a very careful choice of Jetpack version running on the Jetson.

Computer Vision - Unified Game State Vision Model

While performing well with synthetic dataset, extra steps would need to be taken for deployment. Synthetic data often include unrealistic artifacts therefore fine tuning on real world dataset would be essential. Our synthetic dataset does not include opponent entities, but adding opponent objects to Blender should be straight forward. Due to the heavy computational workload of the Attention mechanism, our model is slow and runs at 2FPS even with specialised CUDA kernels for Deformable Attention and vanilla Attention. Future work can reduce capacity for faster inference, or explore other lightweight alternatives. Reducing original image resolution could also be a promising direction but require extensive experimentation. Lighter and faster alternatives can be used to potentially remove the need for the additional YOLO module.

The current model implementation does not consider temporal information. Temporal information can in theory reduce prediction error from occlusion. There are two common methods:

1. Natively train the neural network to input historical states (can be BEV planes or images). This method will increase computational overhead quadratically if it uses a Transformer. LSTM-based implementation would reduce inference overhead but would increase training time.
2. Use Bayesian Updating. This will require detailed manually parameter tuning and might not generalise to all opponents, but extremely efficient at runtime.

Lastly, formulating additional evaluation metrics that account occlusion can give more nuance interpretation to the model's performance.

Strategy AI

Due to the constraints of physical robot development, we were unable to deploy the strategy model on the actual hardware. This prevented us from identifying specific discrepancies between our simulation and the real world or refining our reward shaping based on physical performance. Therefore, **1)** future work must prioritize deploying the model in a real-world environment. Additionally, while TrueSkill ratings indicate the model is performing intelligently, **2)** more comprehensive evaluation techniques, such as playing against human opponents, are necessary to fully uncover its strategic strengths and weaknesses. Also, because the VEX Robotics Competition games are constantly evolving, **3)** the reward shaping and model parameters will need to be updated. Furthermore, **4)** utilizing high-fidelity simulations such as MuJoCo or IsaacSim should be considered to improve the accuracy of the training environment.

References

Balduzzi, David, et al. “Open-Ended Learning in Symmetric Zero-Sum Games.” arXiv:1901.08106, arXiv, 13 May 2019. arXiv.org, https://doi.org/10.48550/arXiv.1901.08106.

Baur, Stefan, et al. “LISO: Lidar-Only Self-Supervised 3D Object Detection.” arXiv:2403.07071, arXiv, 11 Mar. 2024. arXiv.org, https://doi.org/10.48550/arXiv.2403.07071.

Blender Online Community. “Blender - a 3D Modelling and Rendering Package.” Blender Foundation, 2025. https://www.blender.org.

Carion, Nicolas, et al. “End-to-End Object Detection with Transformers.” arXiv:2005.12872, arXiv, 28 May 2020. arXiv.org, https://doi.org/10.48550/arXiv.2005.12872.

Caesar, Holger, et al. “nuScenes: A Multimodal Dataset for Autonomous Driving.” 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020. https://doi.org/10.1109/CVPR42600.2020.01164.

Campbell, Murray, et al. “A comparison of minimax tree search algorithms,” Artificial Intelligence, vol. 20, no. 4, pp. 347–367, Jul. 1983, https://doi.org/10.1016/0004-3702(83)90001-2.

Dalal, N., and B. Triggs. “Histograms of Oriented Gradients for Human Detection.” 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), 2005. https://doi.org/10.1109/CVPR.2005.177.

Dellaert, Frank, et al. “Monte Carlo Localization for Mobile Robots.” Proceedings of the 1999 IEEE International Conference on Robotics and Automation (ICRA), 1999. https://doi.org/10.1109/ROBOT.1999.772544.

Dosovitskiy, Alexey, et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.” arXiv:2010.11929, arXiv, 22 Oct 2020. https://doi.org/10.48550/arXiv.2010.11929.

Everingham, Mark, et al. “The PASCAL Visual Object Classes (VOC) Challenge.” International Journal of Computer Vision 88, no. 2 (2010): 303–338. https://doi.org/10.1007/s11263-009-0275-4.

Felzenszwalb, Pedro F., et al. “A Discriminatively Trained, Multiscale, Deformable Part Model.” 2008 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'08), 2008. https://doi.org/10.1109/CVPR.2008.4587597.

Fox, Dieter. “KLD-Sampling: Adaptive Particle Filters.” Advances in Neural Information Processing Systems 14 (NIPS 2001), 2001. https://doi.org/10.5555/2980539.2980632.

Geiger, Andreas, et al. “Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite.” 2012 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2012. https://doi.org/10.1109/CVPR.2012.6248074.

Girshick, Ross. “Fast R-CNN.” 2015 IEEE International Conference on Computer Vision (ICCV), 2015. https://doi.org/10.1109/ICCV.2015.169.

He, Kaiming, et al. “Deep Residual Learning for Image Recognition.” arXiv:1512.03385, arXiv, 10 Dec. 2015. arXiv.org, https://doi.org/10.48550/arXiv.1512.03385.

Jocher, Glenn, et al. “Ultralytics YOLOv8.” 2023. https://github.com/ultralytics/ultralytics.

Macenski, Steven, et al. "Impact of ROS 2 Node Composition in Robotic Systems." IEEE Robotics and Autonomous Letters (RA-L) (2023). 2023. https://doi.org/10.48550/arXiv.2305.09933.

Macenski, Steven, et al. "Robot Operating System 2: Design, architecture, and uses in the wild." *Science Robotics* 7, no. 66 (2022): eabm6074. 2022. https://doi.org/10.1126/scirobotics.abm6074.

Kumar, Abhinav. “Generalizing Monocular 3D Object Detection.” arXiv:2508.19593, arXiv, 27 Aug. 2025. arXiv.org, https://doi.org/10.48550/arXiv.2508.19593.

Krizhevsky, Alex, et al. “ImageNet Classification with Deep Convolutional Neural Networks.” Advances in Neural Information Processing Systems 25 (NIPS 2012), 2012. https://doi.org/10.1145/3065386.

Lang, Alex H., et al. “PointPillars: Fast Encoders for Object Detection from Point Clouds.” arXiv:1812.05784, arXiv, 7 May 2019. arXiv.org, https://doi.org/10.48550/arXiv.1812.05784.

Land, Edwin H. “The Retinex Theory of Color Vision.” Scientific American 237 (1977): 108–128. https://doi.org/10.1038/scientificamerican1277-108.

Li, Zhiqi, et al. “BEVFormer: Learning Bird’s-Eye-View Representation from Multi-Camera Images via Spatiotemporal Transformers.” arXiv:2203.17270, arXiv, 13 July 2022. arXiv.org, https://doi.org/10.48550/arXiv.2203.17270.

Lin, Tsung-Yi, et al. “Focal Loss for Dense Object Detection.” arXiv:1708.02002, arXiv, 7 Feb. 2018. arXiv.org, https://doi.org/10.48550/arXiv.1708.02002.

Lin, Tsung-Yi, et al. “Microsoft COCO: Common Objects in Context.” European Conference on Computer Vision (ECCV), 2014. https://doi.org/10.1007/978-3-319-10602-1_48.)

Lin, Tsung-Yi, et al. “Feature Pyramid Networks for Object Detection.” arXiv:1612.03144, arXiv, 19 Apr. 2017. arXiv.org, https://doi.org/10.48550/arXiv.1612.03144.

Liu, Wei, et al. “SSD: Single Shot MultiBox Detector.” arXiv:1512.02325, arXiv, 29 Dec. 2016. arXiv.org, https://doi.org/10.48550/arXiv.1512.02325.

Macenski, Steve, et al. "From the Desks of ROS Maintainers: A Survey of Modern & Capable Mobile Robotics Algorithms in the Robot Operating System 2." arXiv:2307.15236, arXiv, 28 Jul. 2023. arXiv.org, https://doi.org/10.48550/arXiv.2307.15236 .

Mathieu, Michaël, et al. “AlphaStar Unplugged: Large-Scale Offline Reinforcement Learning.” arXiv:2308.03526, arXiv, 7 Aug. 2023. arXiv.org, https://doi.org/10.48550/arXiv.2308.03526.

Mousavian, Arsalan, et al. “3D Bounding Box Estimation Using Deep Learning and Geometry.” arXiv:1612.00496, arXiv, 10 Apr. 2017. arXiv.org, https://doi.org/10.48550/arXiv.1612.00496.

Neubeck, A., and L. Van Gool. “Efficient Non-Maximum Suppression.” 18th International Conference on Pattern Recognition (ICPR'06), 2006. https://doi.org/10.1109/ICPR.2006.479.

OpenAI, et al. “Dota 2 with Large Scale Deep Reinforcement Learning.” arXiv:1912.06680, arXiv, 13 Dec. 2019. arXiv.org, https://doi.org/10.48550/arXiv.1912.06680.

Open Source Robotics Foundation. "Gazebo." 2002. https://gazebosim.org.

Philion, Jonah, and Sanja Fidler. “Lift, Splat, Shoot: Encoding Images From Arbitrary Camera Rigs by Implicitly Unprojecting to 3D.” arXiv:2008.05711, arXiv, 13 Aug. 2020. arXiv.org, https://doi.org/10.48550/arXiv.2008.05711.

Redmon, Joseph, et al. “You Only Look Once: Unified, Real-Time Object Detection.” arXiv:1506.02640, arXiv, 9 May 2016. arXiv.org, https://doi.org/10.48550/arXiv.1506.02640.

Ren, Shaoqing, et al. “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks.” arXiv:1506.01497, arXiv, 6 Jan. 2016. arXiv.org, https://doi.org/10.48550/arXiv.1506.01497.

Ross, Stephane, et al. “A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning.” arXiv:1011.0686, arXiv, 16 Mar. 2011. arXiv.org, https://doi.org/10.48550/arXiv.1011.0686.

Schulman, John, et al. “Proximal Policy Optimization Algorithms.” arXiv:1707.06347, arXiv, 28 Aug. 2017. arXiv.org, https://doi.org/10.48550/arXiv.1707.06347.

Schulman, John, et al. “Trust Region Policy Optimization.” arXiv:1502.05477, arXiv, 20 Apr. 2017. arXiv.org, https://doi.org/10.48550/arXiv.1502.05477.

Shi, Guangsheng, et al. “Pillar R-CNN for Point Cloud 3D Object Detection.” arXiv:2302.13301, arXiv, 26 Feb. 2023. arXiv.org, https://doi.org/10.48550/arXiv.2302.13301.

Shi, Shaoshuai, et al. “PV-RCNN: Point-Voxel Feature Set Abstraction for 3D Object Detection.” arXiv:1912.13192, arXiv, 9 Apr. 2021. arXiv.org, https://doi.org/10.48550/arXiv.1912.13192.

Silver, David, et al. “Mastering the game of Go with deep neural networks and tree search”. Nature 529, 484–489 (2016). https://doi.org/10.1038/nature16961.

Tabor, Griffin, Jonathan Berry, and Evan Gilgenbach. “Adversarial Strategic Games and Robotic Design.” Major Qualifying Project, Worcester Polytechnic Institute, 2018. https://digital.wpi.edu/pdfviewer/6q182n05h.

Thrun, Sebastian, et al. “Robust Monte Carlo Localization for Mobile Robots.” Artificial Intelligence 128, no. 1-2 (2001): 99–141. https://doi.org/10.1016/S0004-3702(01)00069-8.

Tian, Zhi, et al. “FCOS: Fully Convolutional One-Stage Object Detection.” arXiv:1904.01355, arXiv, 20 Aug. 2019. arXiv.org, https://doi.org/10.48550/arXiv.1904.01355.

Vaswani, Ashish, et al. “Attention Is All You Need.” arXiv:1706.03762, arXiv, 2 Aug. 2023. arXiv.org, https://doi.org/10.48550/arXiv.1706.03762.

VEX Robotics Inc. "VEX V5 Robotics Competition Push Back 2025-2026 Game Manual Version 4.0" VEX Robotics, 4 April. 2025. vexrobotics.com, https://content.vexrobotics.com/docs/25-26/v5rc-push-back/docs/Push-Back-4.0.pdf.

Vina, Abirami. "YOLO11 on Jetson Orin Nano: Benchmarks & Performance." *Ultralytics Blog*, Ultralytics, 9 Jan. 2025, www.ultralytics.com/blog/ultralytics-yolo11-on-nvidia-jetson-orin-nano-super-fast-and-efficient.

Viola, Paul, and Michael Jones. “Rapid Object Detection Using a Boosted Cascade of Simple Features.” Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2001. https://doi.org/10.1109/CVPR.2001.990517.

Wang, Ao, et al. “YOLOv10: Real-Time End-to-End Object Detection.” arXiv:2405.14458, arXiv, 23 May 2024. https://doi.org/10.48550/arXiv.2405.14458.

Wang, Chien-Yao, et al. “YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information.” arXiv:2402.13616, arXiv, 21 Feb 2024. https://doi.org/10.48550/arXiv.2402.13616.

Wang, Tai, et al. “FCOS3D: Fully Convolutional One-Stage Monocular 3D Object Detection.” arXiv:2104.10956, arXiv, 24 Sept. 2021. arXiv.org, https://doi.org/10.48550/arXiv.2104.10956.

Wang, Yue, et al. “DETR3D: 3D Object Detection from Multi-View Images via 3D-to-2D Queries.” arXiv:2110.06922, arXiv, 13 Oct. 2021. arXiv.org, https://doi.org/10.48550/arXiv.2110.06922.

Watkins, Christopher, er al. “Q-learning.” Mach Learn 8, 279–292 (1992). https://doi.org/10.1007/BF00992698

Williams, Ronald, “Simple statistical gradient-following algorithms for connectionist reinforcement learning.” Mach Learn 8, 229–256 (1992). https://doi.org/10.1007/BF00992696.

Yin, Tianwei, et al. “Center-Based 3D Object Detection and Tracking.” arXiv:2006.11275, arXiv, 6 Jan. 2021. arXiv.org, https://doi.org/10.48550/arXiv.2006.11275.

"YOLO11 vs YOLOv10: A Comprehensive Technical Comparison of Real-Time Object Detectors." *Ultralytics YOLO Docs*, Ultralytics, 28 Feb. 2026, docs.ultralytics.com/compare/yolo11-vs-yolov10/ .

Zietek, Jacob, et al. "Pac-Man Pete: An Extensible Framework for Building AI in VEX Robotics." arXiv:2211.14385, arXiv, 25 Nov. 2022. arXiv.org, https://doi.org/10.48550/arXiv.2211.14385 .

Zhao, Zhong-Qiu, et al. “Object Detection with Deep Learning: A Review.” arXiv:1807.05511, arXiv, 16 Apr. 2019. arXiv.org, https://doi.org/10.48550/arXiv.1807.05511.

Zhou, Yin, and Oncel Tuzel. “VoxelNet: End-to-End Learning for Point Cloud Based 3D Object Detection.” arXiv:1711.06396, arXiv, 17 Nov. 2017. arXiv.org, https://doi.org/10.48550/arXiv.1711.06396.

Zhu, Xizhou, et al. “Deformable DETR: Deformable Transformers for End-to-End Object Detection.” arXiv:2010.04159, arXiv, 18 Mar. 2021. arXiv.org, https://doi.org/10.48550/arXiv.2010.04159.